Análisis y Diseño de Algoritmos
I

Valeria Bengolea
Departamento de Computación
Facultad de Ciencias Exactas, Físico-Químicas y Naturales
Universidad Nacional de Río Cuarto

Algoritmos Greedy

Notas basadas en el material preparado por el Dr. Nazareno Aguirre

Estrategias de Diseño de Algoritmos

Las estrategias de diseño de algoritmos nos brindan herramientas para atacar el problema de construir soluciones algorítmicas para problemas. Hemos visto ya algunas de estas estrategias, tales como Fuerza Bruta, Divide & Conquer, Decrease & Conquer y Programación Dinámica (también mediante Memoization).

Hasta el momento, hemos utilizado estas técnicas para construir soluciones algorítmicas exactas para diferentes problemas. Veremos ahora una nueva técnica, denominada Técnica Greedy, que si bien puede usarse para resolver problemas de forma exacta, tiene una enorme aplicación en la construcción de soluciones algorítmicas aproximadas para problemas.

Técnica Greedy

Una forma general de describir la técnica Greedy es la siguiente:

Dado un problema P, construya una solución (no necesariamente exacta) para P basada en la siguiente estrategia:

> Para resolver el problema P, se realizará una secuencia de pasos. En cada paso se "expandirá" una solución parcial construida hasta ese momento que sea localmente óptima, en el sentido de que, con la información disponible en el momento, sea la mejor solución entre todas las posibilidades localmente visibles.

Técnica Greedy

Más precisamente, cada paso de "expansión" de la solución parcial construida debe dar:

- Una solución localmente optimal,
- Un paso hacia la solución global al problema que sea irrevocable (i.e., no puede ser revisado más adelante en el cómputo de la solución global)

Soluciones óptimas vs Soluciones aproximadas de un problema

Ya vimos soluciones aproximadas a algunos problemas en las primeras clases de la materia:

- Coloreo de grafo
- Dado un conjunto P de puntos del plano, elegir una forma de recorrerlos de manera que la distancia global recorrida sea la mínima

Ejemplo: Coloreo de Grafos

El problema de coloreo de grafos consiste en lo siguiente:

> Dado un grafo G, asignar la mínima cantidad de colores a los nodos de manera tal que, si dos nodos x e y son adyacentes en G, reciban colores diferentes.

Al igual que el Problema del Agente Viajero, este problema pertenece a la clase de problemas NP-completos. Para estos problemas, las únicas soluciones óptimas conocidas son del tipo "probar todas las posibilidades".

En nuestro caso, probar todas las posibilidades consiste en probar con todas las posibles asignaciones de colores a nodos, comenzando con un color, luego probando con dos, y así sucesivamente hasta conseguir una coloración válida.

Una Propuesta para el Problema de Coloreo de Grafos

Una propuesta razonable para resolver el problema de coloreo de grafos con una cantidad mínima de colores es la siguiente:

> Comenzar a visitar los nodos en orden, coloreando todos los nodos que se pueda con un color, sin causar conflictos entre nodos adyacentes. Si quedan nodos sin colorear, tomar un nuevo color y repetir el coloreo de tantos nodos no coloreados como se pueda, nuevamente sin causar conflictos. Continuar el proceso con nuevos colores hasta que ya no queden nodos sin colorear.

Este procedimiento corresponde a la técnica de diseño de algoritmos greedy

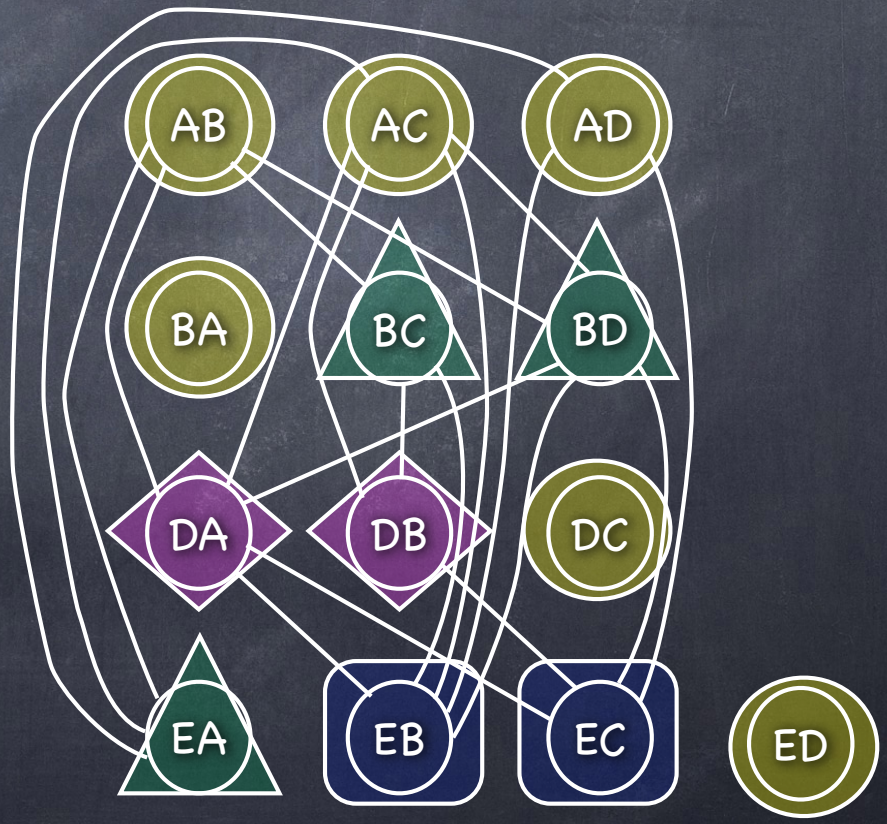
Una Propuesta para el Problema de Coloreo de Grafos

Veamos un ejemplos de cómo funciona la propuesta descripta:

Comenzar a visitar los nodos en orden, coloreando todos los nodos que se pueda con un color, sin causar conflictos entre nodos adyacentes.

Si quedan nodos sin colorear, tomar un nuevo color y repetir el coloreo de tantos nodos no coloreados como se pueda, nuevamente sin causar conflictos.

Continuar el proceso con nuevos colores hasta que ya no queden nodos sin colorear.



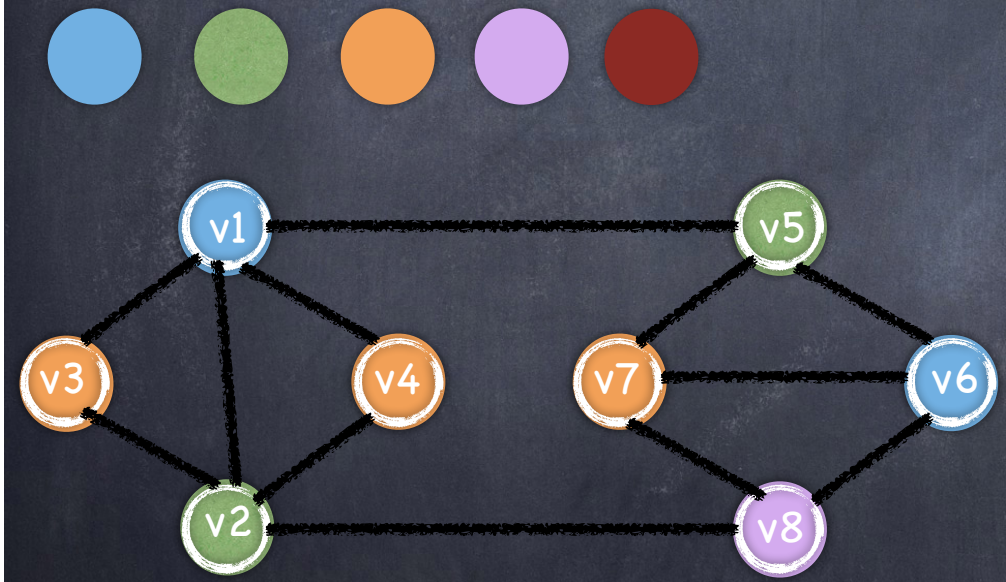
Una Propuesta para el Problema de Coloreo de Grafos

El procedimiento más importante de la propuesta descripta es el que construye conjuntos de nodos coloreables con un mismo color. Una descripción abstracta, en pseudo-código, de este proceso es la siguiente:

```
function graph_colouring (var g: Graph): SET
  begin
    newclr:= empty_set;
    for cada nodo no coloreado v de g do
      if v no es adyacente a ningún vértice en newclr then
        begin
          marcar v como coloreado;
          agregar v a newclr;
        end
    end
    graph_colouring := newclr
  end
```

Heurística Greedy para el Coloreo de Grafos

El siguiente ejemplo muestra que la solución al problema del coloreo de grafos que propusimos no es óptima.

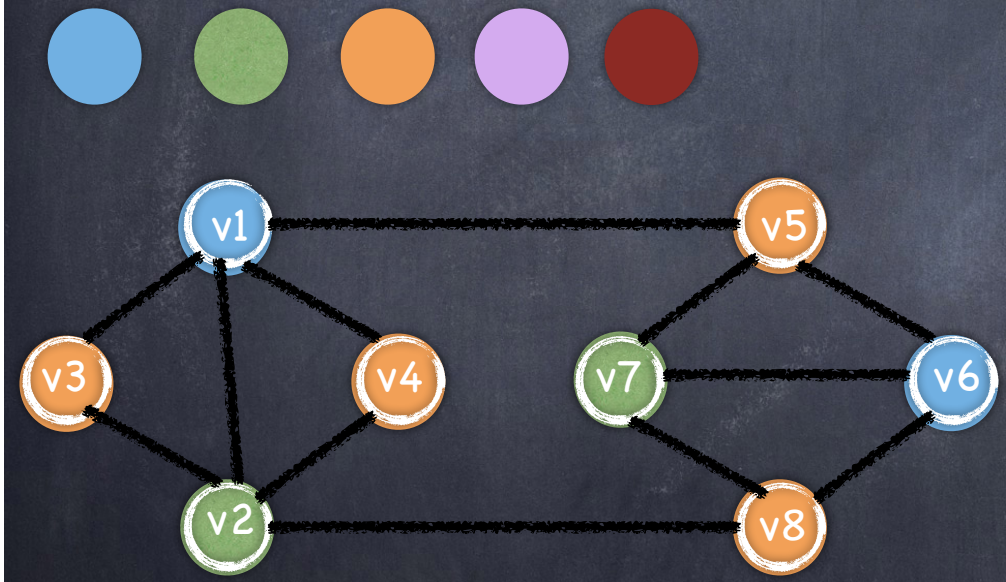


Orden de visita

[v1, v2, v3, v4, v5, v6, v7, v8]

4 colores!

Heurística Greedy para el Coloreo de Grafos



Orden de visita

[v1, v6, v2, v7, v5, v3, v4, v8]

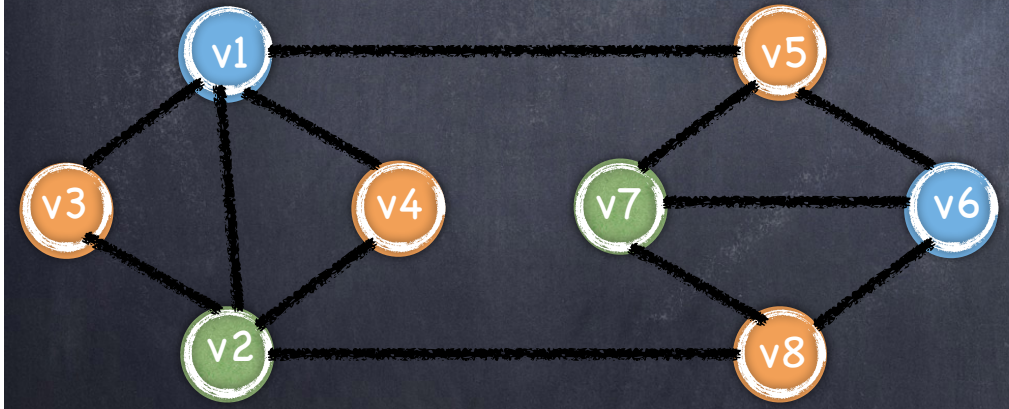
3 colores!

Heurí… …para el

Dado un orden "correcto", el algoritmo greedy produce una solución óptima (pero hay que conocer cuál es ese orden)

… visita

[v1, v6, v2, v7, v5, v3, v4, v8]



3 colores!

Ejemplo: Problema de dar Cambio

Consideremos el problema de dar cambio por C centavos, usando monedas de valores:

$$d_1 > d_2 > \ldots > d_k$$

de manera tal que el número de monedas dadas sea minimal.

Ejemplo: Problema de dar Cambio

La solución greedy descripta anteriormente es óptima. Esto se debe a una propiedad particular de los valores asociados con las monedas, pero no se cumple en todos los casos.
Consideremos por ejemplo la siguiente instancia del problema de dar cambio:

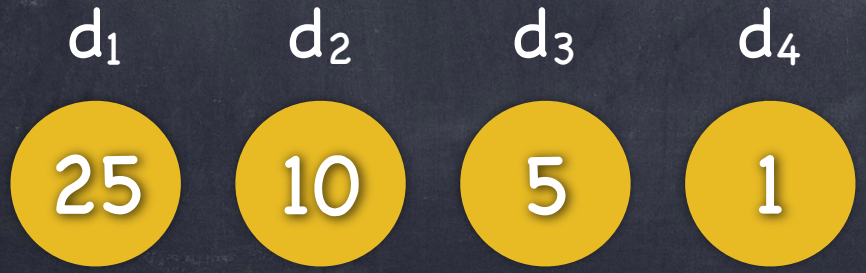
$$C = 25$$



Ejemplo: Problema de dar Cambio

Supongamos por ejemplo que tenemos la siguiente instancia de este problema:

$$C = 63$$



Problema de dar Cambio

La solución (greedy) que uno aplicaría sería simplemente la siguiente:

1) Elegir la moneda d_i más grande posible que no exceda el valor C,

2) Incluir d_i en la solución.

3) Reducir el problema a dar cambio por $C - d_i$.

Problema de dar Cambio

En este caso tendríamos la siguiente sucesión de pasos:

C = 63, Cambio = {};

C = 38, Cambio = {25};

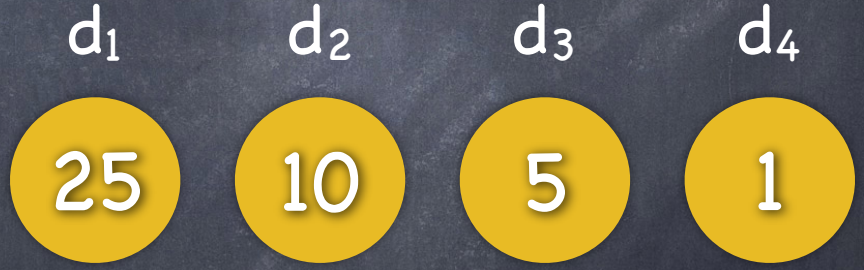
C = 13, Cambio = {25, 25};

C = 3, Cambio = {25, 25, 10};

C = 2, Cambio = {25, 25, 10, 1};

C = 1, Cambio = {25, 25, 10, 1, 1};

C = 0, Cambio = {25, 25, 10, 1, 1, 1}



Problema de dar Cambio

La solución greedy descripta daría por resultado lo siguiente:

C = 25, Cambio = {};

C = 18, Cambio = {7};

C = 11, Cambio = {7, 7};

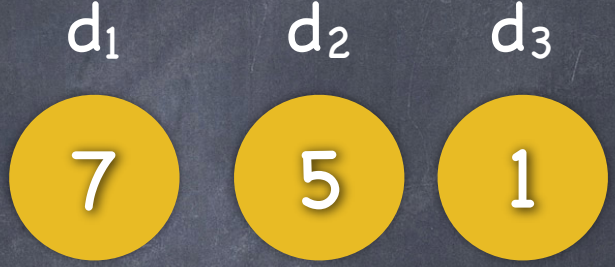
C = 4, Cambio = {7, 7, 7};

C = 3, Cambio = {7, 7, 7, 1};

C = 2, Cambio = {7, 7, 7, 1, 1};

C = 1, Cambio = {7, 7, 7, 1, 1, 1};

C = 0, Cambio = {7, 7, 7, 1, 1, 1, 1}.



Pero {5, 5, 5, 5, 5} es un cambio válido

que requiere menos monedas!

Ejemplo:
Recorrido de impresión en una impresora 3D

Problema del agente Viajero

A modo de ejemplo, ilustrando las diferentes cuestiones a tener en cuenta en la resolución algorítmica de problemas, consideremos el siguiente problema:

> Dado un conjunto P de puntos del plano, elegir una forma de recorrerlos de manera que la distancia global recorrida sea la mínima

Algunas preguntas que se pueden hacer para comprender mejor el problema son las siguientes:

¿hay algún punto de inicio específico? **NO**

¿cómo son los movimientos en el plano? **En línea recta del origen al destino**

¿hay algún punto final específico? **Misma posición que punto de salida**

Recorrido Óptimo de Impresión

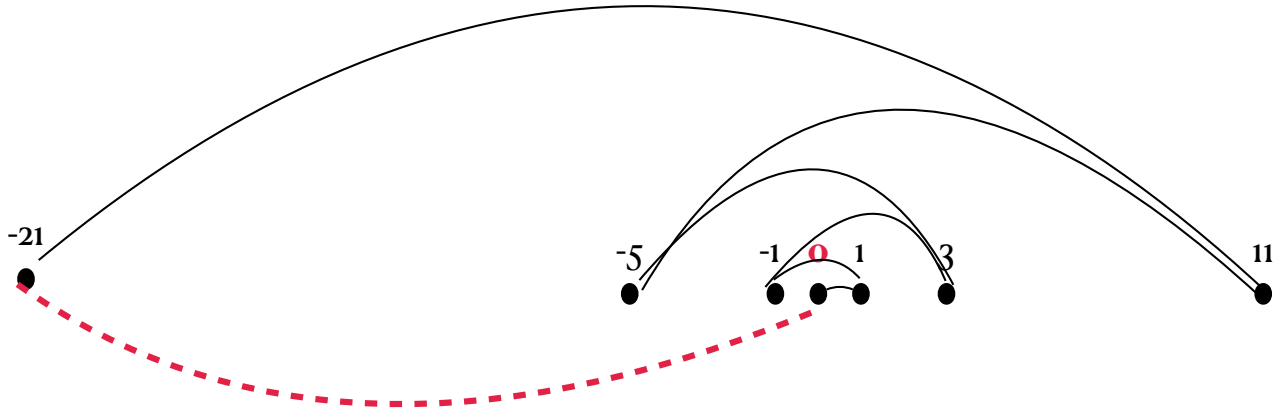
Una propuesta para resolver el problema del recorrido óptimo de impresión es elegir siempre el punto vecino no visitado más próximo al punto actual.

Primera Propuesta

```
RecorridoOptimo(P: Cto. de n puntos en el plano)
    elegir un punto po de P como inicial
    visitar po
    i := 1
    while i<n do
        begin
            elegir punto pi no visitado más próximo a p(i-1)
            visitar pi
          i := i+1
        end
    retornar a po (desde p(n-1))
```

Solución aproximada

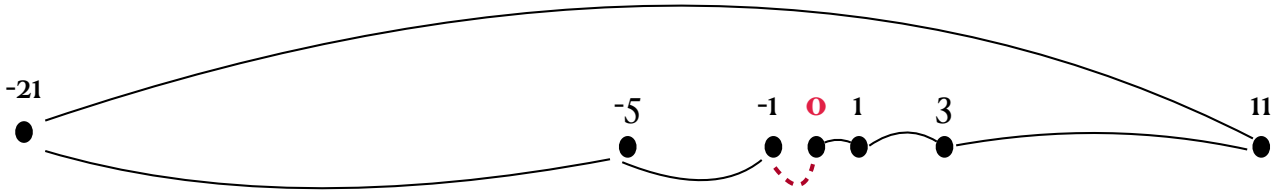
Mala Instancia de Vecino más Próximo



Distancia = 1 + 2 + 4 + 8 + 16 + 32 + 21 = 84

Distancia Óptima = ?

Mala Instancia de Vecino más Próximo



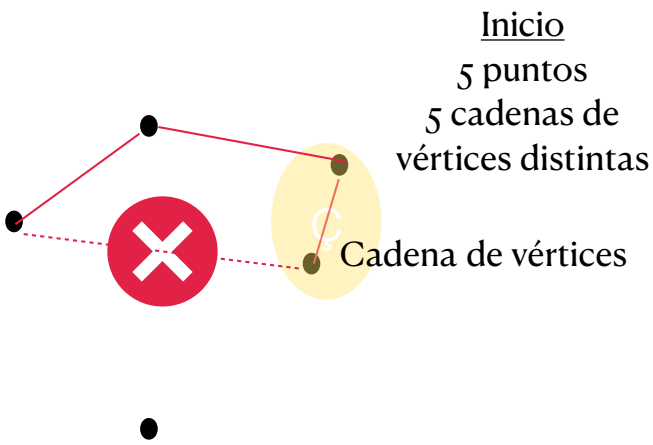
Distancia Óptima = 1 + 2 + 8 + 32 + 16 + 4 +1 = **64**

Recorrido Óptimo de Impresión

Segunda Propuesta

Una segunda propuesta para resolver el problema del recorrido óptimo de impresión es armar el recorrido en base a pares de puntos más próximos entre sí.

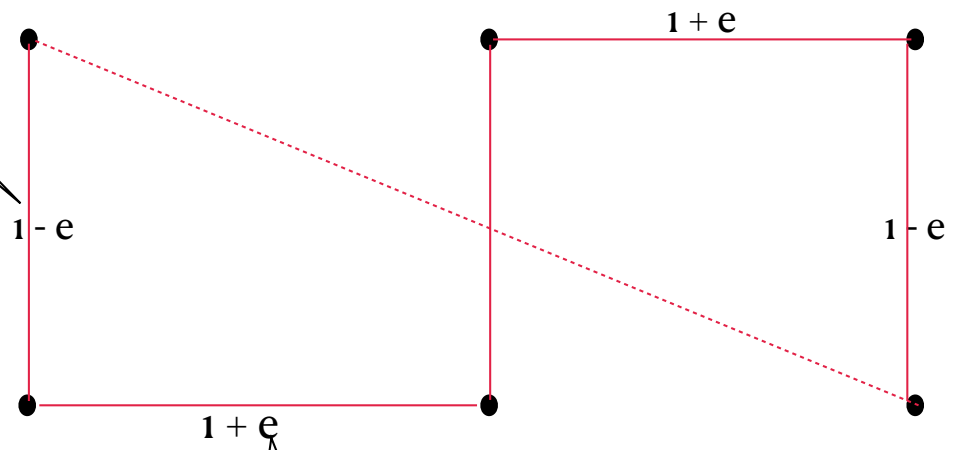
```
RecorridoOptimo'(P: Cto. de n puntos en el plano)
    for i := 1 to n-1 do
        begin
          d := infinity
          for (s,t) en cadenas de vértices distintas do
                begin
                  if dist(s,t) <= d then
                        begin
                            d := dist(s,t)
                            sm := s
                            tm := t
                        end
                  end
              conectar sm y tm con un arco
        conectar las dos puntas con un arco
```



Solución aproximada

Instancia de Pares más Próximos

1 - un epsilon (un



1 + un epsilon (un poquito más de 1)

Distancia global mínima?

Recorrido Óptimo de Impresión

Tercera Propuesta: Solución óptima

Una tercera propuesta, sin dudas correcta, consiste en intentar todas las permutaciones, y quedarse con la mejor.

Fuerza Bruta!

```
RecorridoOptimo''(P: Cto. de n puntos
    en el plano)
    d := infinity
    for cada permutación Pi de P do
        if cost(Pi) <= d then
            begin
                d := cost(Pi)
                Pmin := Pi
            end
    end
    retornar Pmin
```

Desafortunadamente, esta solución es muy costosa en tiempo.

Para 20 puntos, tiene que considerar 2432902008176640000 combinaciones.

Ejemplo: Árbol Abarcador de Costo Mínimo

Consideremos ahora el siguiente problema. Dado un grafo conexo no dirigido G, con costos en los arcos, se desea obtener el árbol abarcador de costo mínimo (minimum spanning tree).

Un árbol abarcador de un grafo conexo G es un subgrafo G' de G que es

- acíclico,
- conexo y
- contiene todos los nodos de G.

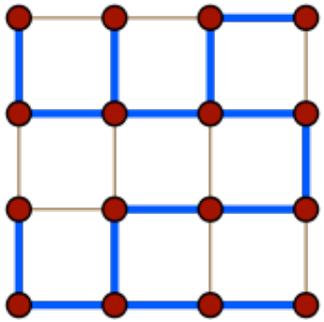
Ejemplo: Árbol Abarcador de Costo Mínimo

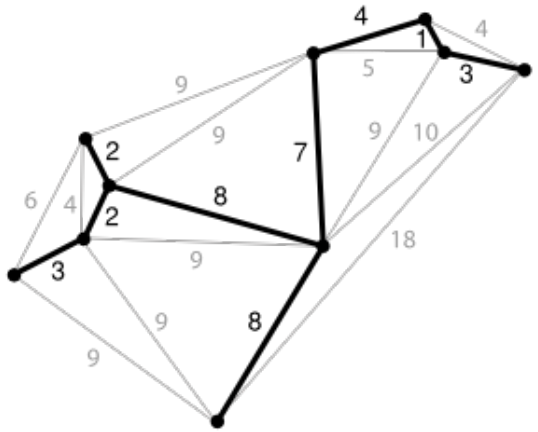
Consideremos ahora el siguiente problema. Dado un grafo conexo no dirigido G, con costos en los arcos, se desea obtener el árbol abarcador de costo mínimo (minimum spanning tree).

Un árbol abarcador de un grafo conexo G es un subgrafo G' de G que es

- acíclico,
- conexo y
- contiene todos los nodos de G.

Ej.:





Ejemplo: Árbol Abarcador de Costo Mínimo (cont.)

Un algoritmo para resolver el problema de computar el árbol abarcador de costo mínimo es el Algoritmo de Prim que consiste en lo siguiente:

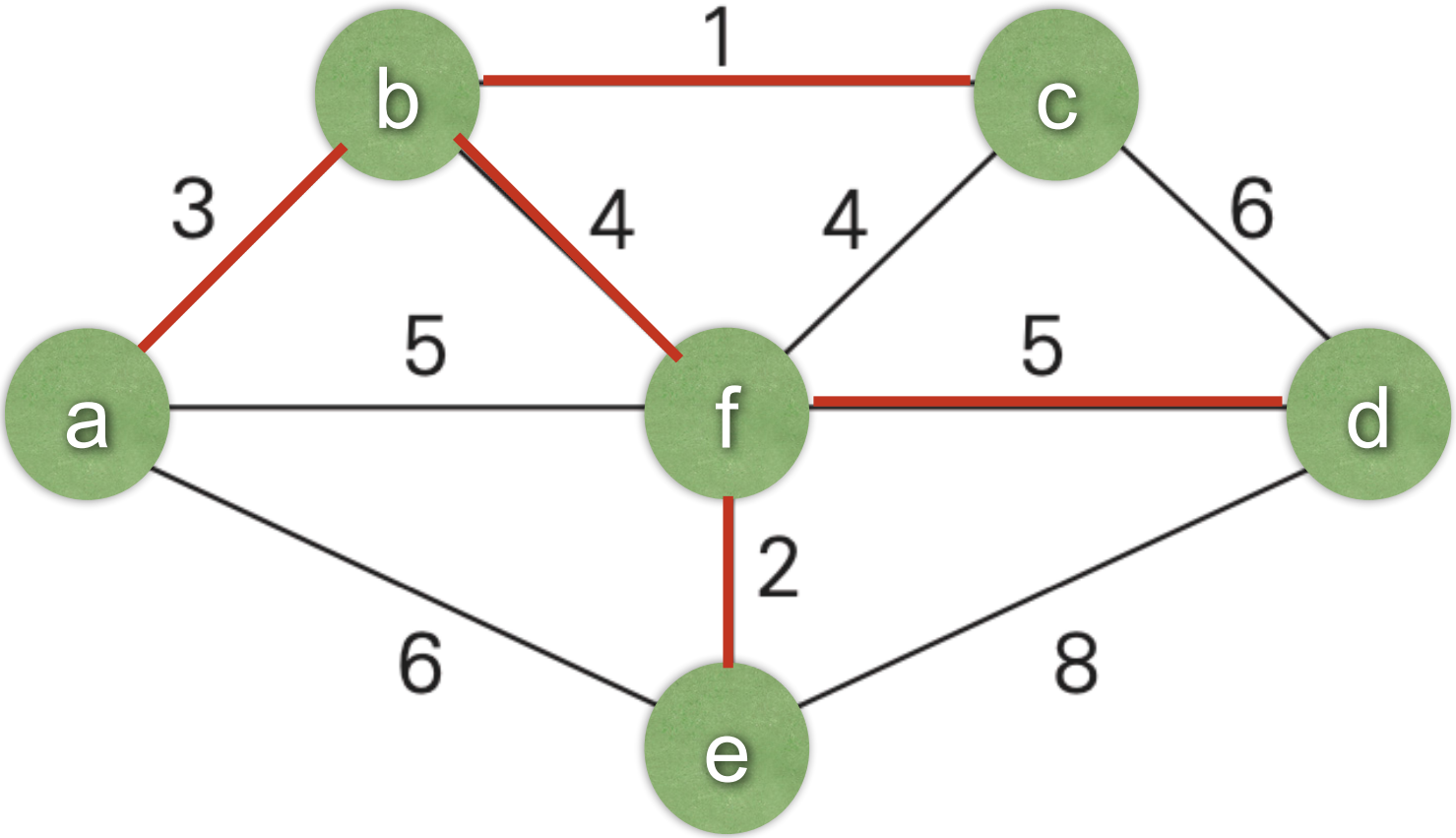
1. Tomar cualquier vértice del grafo G. Esta es la primera solución parcial T1.

2. En cada iteración, construir el árbol T(i+1) agregando a Ti el vértice más cercano a Ti que no pertenece ya a Ti.

3. Terminar cuando ya no queden vértices por agregar.

Claramente, el segundo paso toma decisiones de manera "greedy".

Ejemplo: Árbol Abarcador de Costo Mínimo (cont.)



Ejemplo: Árbol Aba… Mínimo (cont.)

El algoritmo de Prim siempre produce el mínimo spanning tree

```
MST-PRIM(G)
    for each u ∈ G.V
        u.key = ∞
        u.parent = NIL
    root = G.V0   // Elegimos un vértice inicial como la raíz del MST
    root.key = 0
    Q = PriorityQueue(G.V) // ordenando por el valor de 'key'
    while Q ≠ ∅
        u = EXTRACT-MIN(Q)
        for each v ∈ G.Adj[u]
            if v ∈ Q and G.weight(u, v) < v.key
                v.parent = u
                DECREASE-KEY(Q, v, G.weight(u, v))
```

Ejemplo: Árbol Abarcador de Costo Mínimo (cont.)

```
MST-PRIM(G)
    for each u ∈ G.V
        u.key = ∞
        u.parent = NIL
    root = G.V0   // Elegimos un vértice inicial como la raíz del MST
    root.key = 0
    Q = PriorityQueue(G.V) // ordenando por el valor de 'key'
    while Q ≠ ∅
        u = EXTRACT-MIN(Q)
        for each v ∈ G.Adj[u]
            if v ∈ Q and G.weight(u, v) < v.key
                v.parent = u
                DECREASE-KEY(Q, v, G.weight(u, v))
```

O(V)

Se ejecuta V veces

log(V) con heaps

Se ejecuta E veces en TOTAL

O(log V) porque se actualizan las prioridades en Q

Ejemplo: Árbol Abarcador de Costo Mínimo (cont.)

$$T(V,E) = V + V\,.\,logV + E\,.\,logV = (V+E)\ log\ V$$

$$\in O(V + E\,.\,log\ V)$$

Ejemplo



Estado Inicial

El primer bucle for each pone a todos con key = inf y parent = NIL

Elegimos la raíz: root = 1, por lo tanto 1.key = 0.

Q **inicial:** [1:0, 2:inf, 3:inf, 4:inf, 5:inf]

Iteración 1 del while

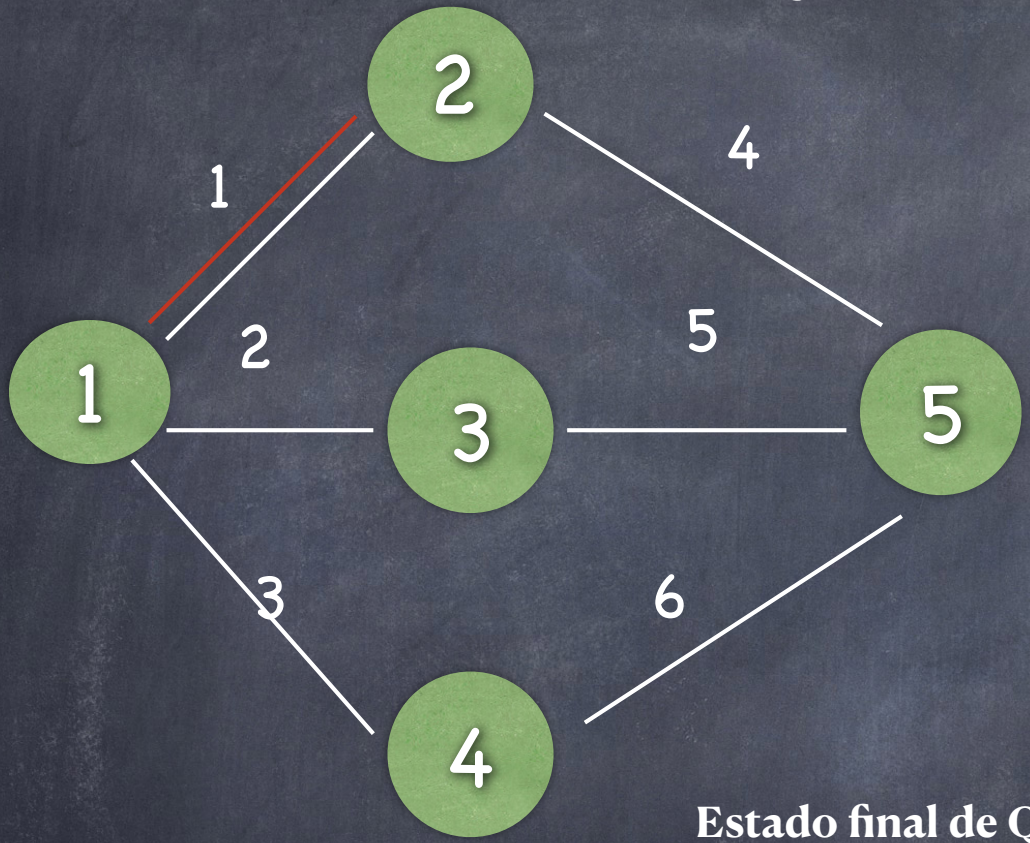
EXTRACT-MIN(Q) Sale el nodo **1**

Estado de Q: [2:inf, 3:inf, 4:inf, 5:inf]

Miramos los vecinos del nodo 1: **(2, 3, 4)**

Estado final de Q en la Iteración 1: [2:1, 3:2, 4:3, 5:inf]

Ejemplo

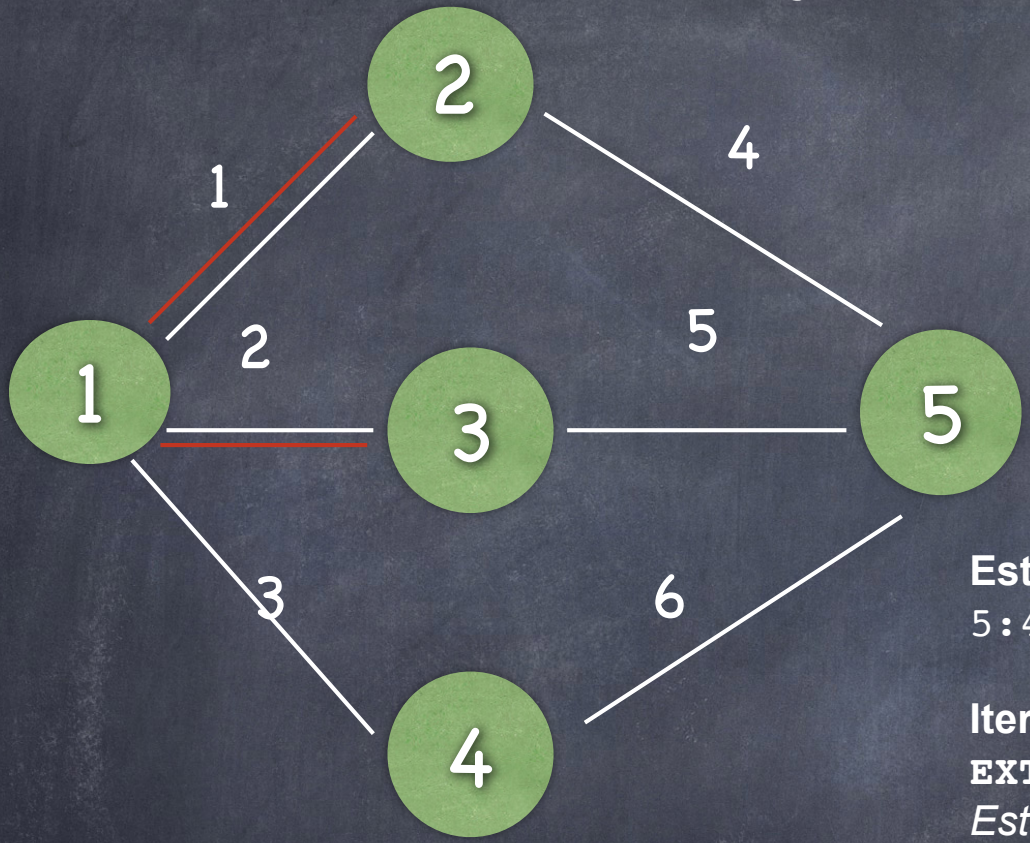


Estado final de Q en la Iteración 1: [2:1, 3:2, 4:3, 5:inf]

Iteración 2 del while
EXTRACT-MIN(Q) Sale el nodo **2** (su key es 1).
Estado de Q: [3:2, 4:3, 5:inf]

Bucle for each v ∈ G.Adj[2] (Miramos vecinos del nodo 2, **solo el 5**):
Estado final de Q en la Iteración 2: [3:2, 4:3, 5:4]

Ejemplo



Estado final de Q en la Iteración 2: `[ 3:2, 4:3, 5:4 ]`

Iteración 3 del `while`
`EXTRACT-MIN(Q)` Sale el nodo **3** (su `key` es 2).
Estado de Q: `[ 4:3, 5:4 ]`

Bucle `for each v ∈ G.Adj[3]` (Miramos vecinos del nodo 3):

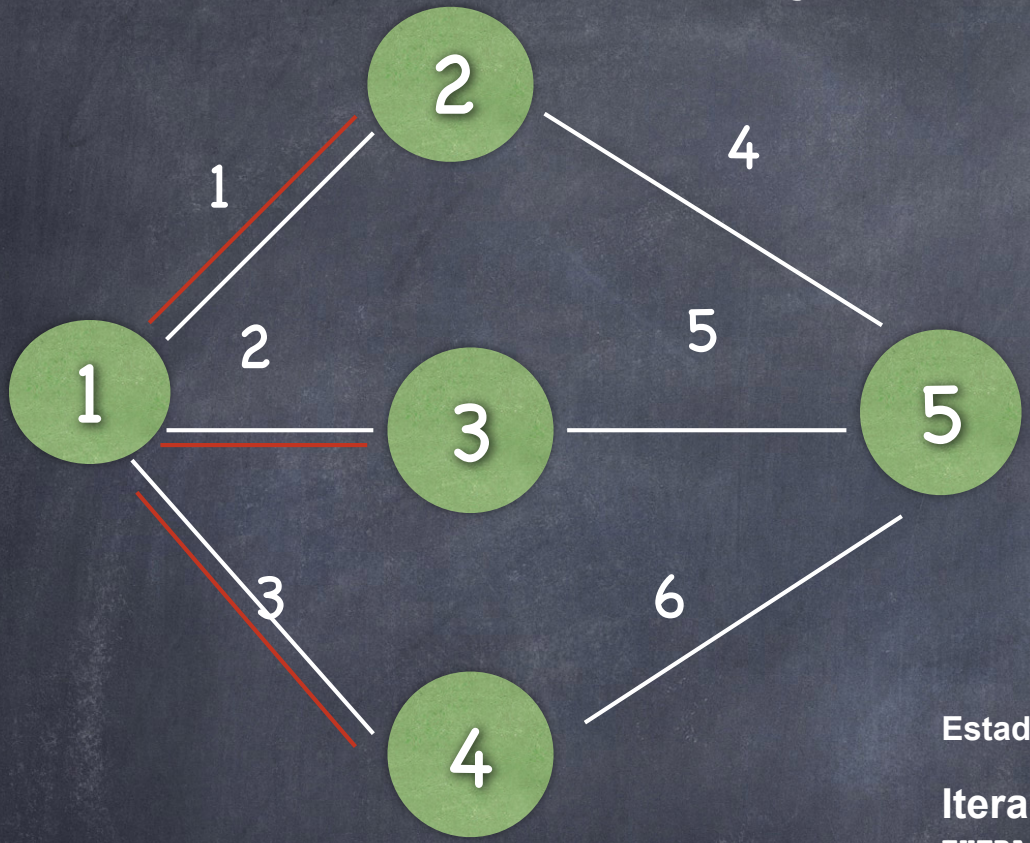
Vecino 1: Ya no está en Q. Se ignora.

Vecino 5: Está en Q y el peso es 5. ¿Es 5 < 5.key? **FALSO**.

No se hace nada.

Estado final de Q en la Iteración 3: `[ 4:3, 5:4 ]`

Ejemplo



Estado final de Q en la Iteración 3: `[ 4:3, 5:4 ]`

Iteración 4 del `while`

`EXTRACT-MIN(Q)` Sale el nodo **4** (su `key` es 3).

Estado de Q `[ 5:4 ]`

Bucle `for each v ∈ G.Adj[4]` (Miramos vecinos del nodo 4):

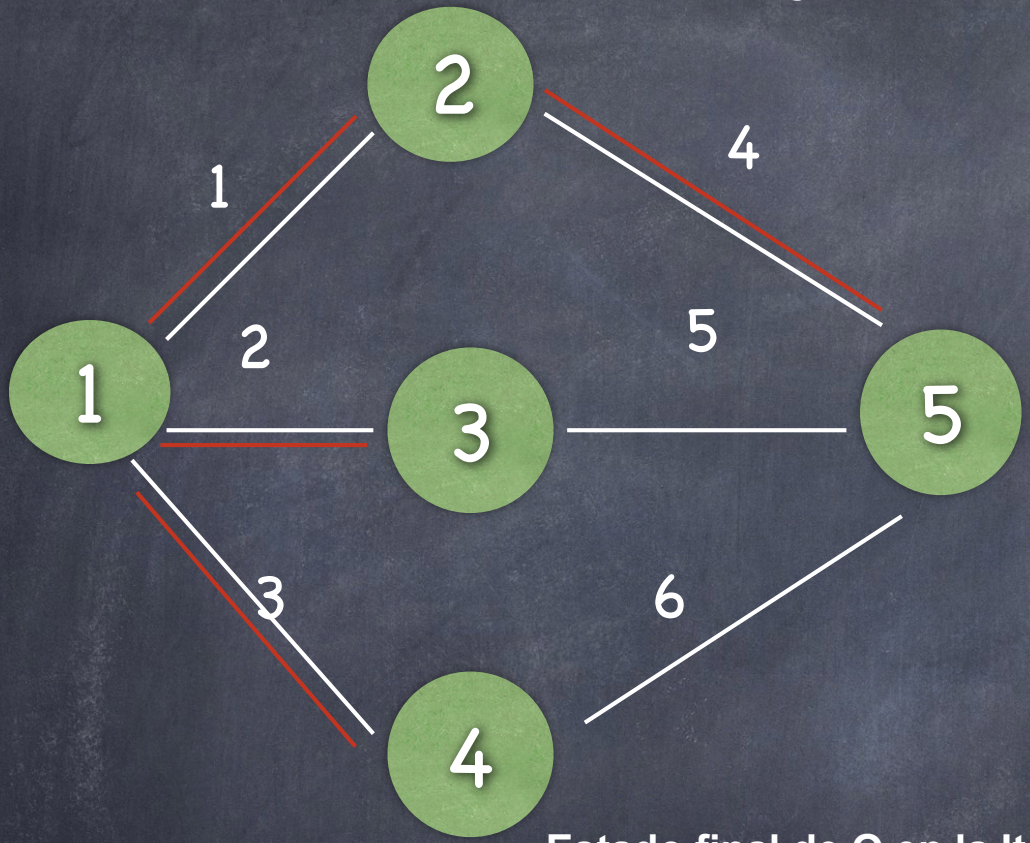
Vecino 1: Ya no está en Q. Se ignora.

Vecino 5: Está en Q y el peso es 6. ¿Es 6 < 5.key? **FALSO**.

Tampoco se hace nada.

Estado final de Q en la Iteración 4: `[ 5:4 ]`

Ejemplo



Estado final de Q en la Iteración 4: `[ 5:4 ]`

Iteración 5 del `while`
`EXTRACT-MIN(Q)` Sale el nodo **5** (su `key` es 4).

Estado de Q: `[ ]`

Bucle `for each v ∈ G.Adj[5]`: Todos sus vecinos (2, 3, 4) ya salieron de Q.
El bucle no hace nada.
Fin del algoritmo.

Ejemplo: Árbol Abarcador de Costo Mínimo (cont.)

Otro algoritmo Greedy para computar el árbol abarcador de costo mínimo es el Algoritmo de Kruskal. Éste consiste en lo siguiente:

1. Ordenar los arcos según sus costos (de forma no decreciente).

2. Comenzar con el arco (y los vértices involucrados) de mínimo costo. Esto constituye la solución parcial inicial T1.

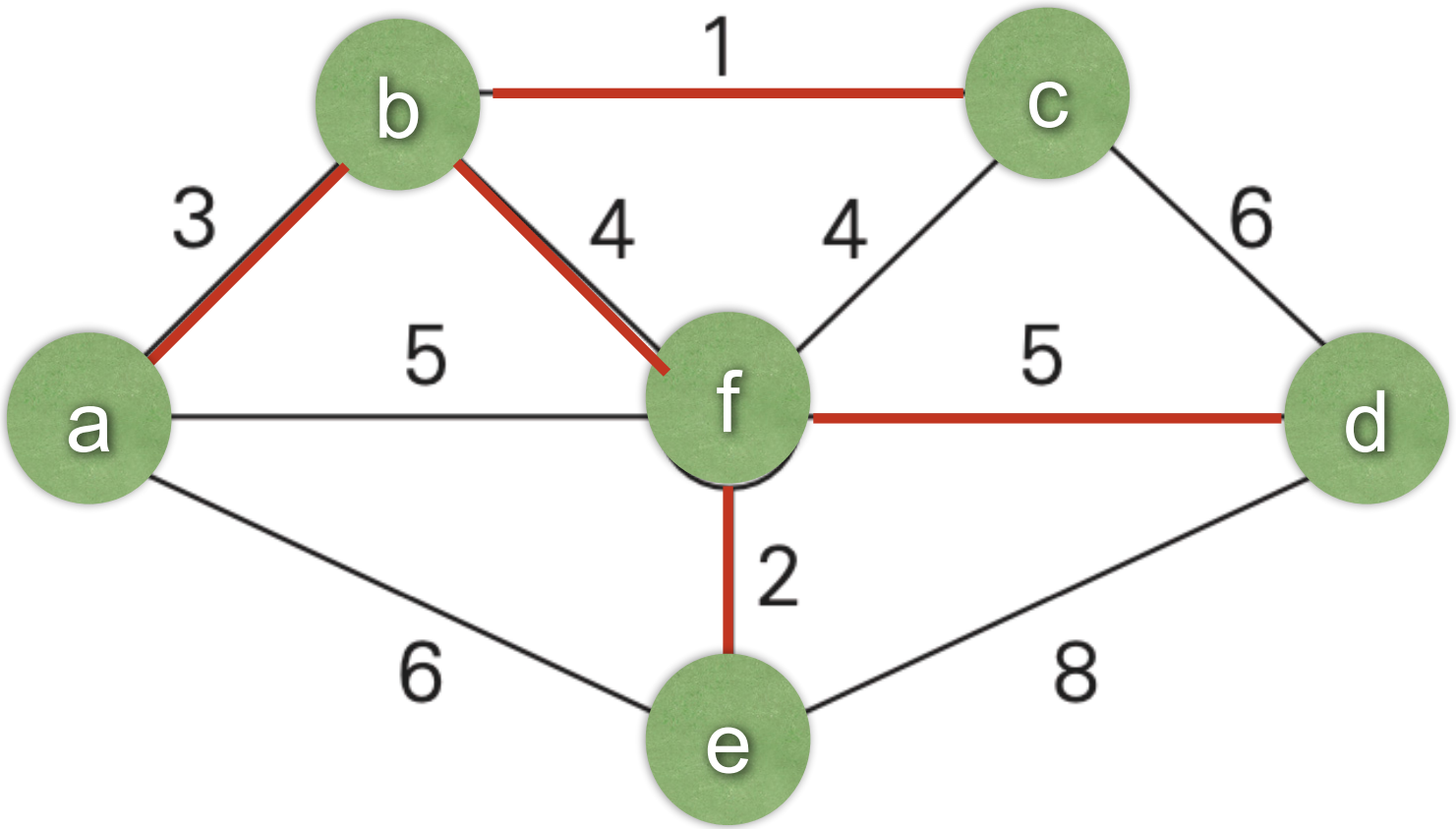
3. En cada iteración, construir T(i+1) agregando el arco de menor costo que no pertenezca a Ti y que no genere un ciclo.

4. Terminar cuando Ti sea conexo e involucre todos los nodos de G.

Nuevamente, el paso 3 corresponde a una decisión Greedy.

Ejemplo: Árbol Abarcador de Costo Mínimo (cont.)

bc	ef	ab	bf	cf	af	df	ae	cd	de
1	2	3	4	4	5	5	6	6	8



Ejemplo: Árbol Abarcador de Costo Mínimo (cont.)

ALGORITHM *Kruskal*(G)
//Kruskal's algorithm for constructing a minimum spanning tree
//Input: A weighted connected graph $G = \langle V , E \rangle$
//Output: E_T , the set of edges composing a minimum spanning tree of
//G sort E in nondecreasing order of the edge weights $w(e_{i_1}) \le \cdots \le w(e_{i_{|E|}})$
$E_T \leftarrow \emptyset$;
ecounter $\leftarrow 0$
k $\leftarrow 0$
while ecounter $< |V| - 1$ **do**
k$\leftarrow$k+1
if $E_T \cup \{e_{i_k}\}$ is acyclic
$E_T \leftarrow E_T \cup \{e_{i_k}\}$;
ecounter $\leftarrow$ ecounter + 1
return E_T

Para verificar si ET ∪ {e_ik} es acíclico, si ejecutamos un recorrido DFS(G') o BFS(G') sobre el árbol parcial para ver si u y v ya están conectados.
Costo por cada arista evaluada: O(V)

Disjoint sets

Los disjoint sets mantienen clases de equivalencia. Ej: Al inicio 8 clases: 0, 1, 2,....,7.

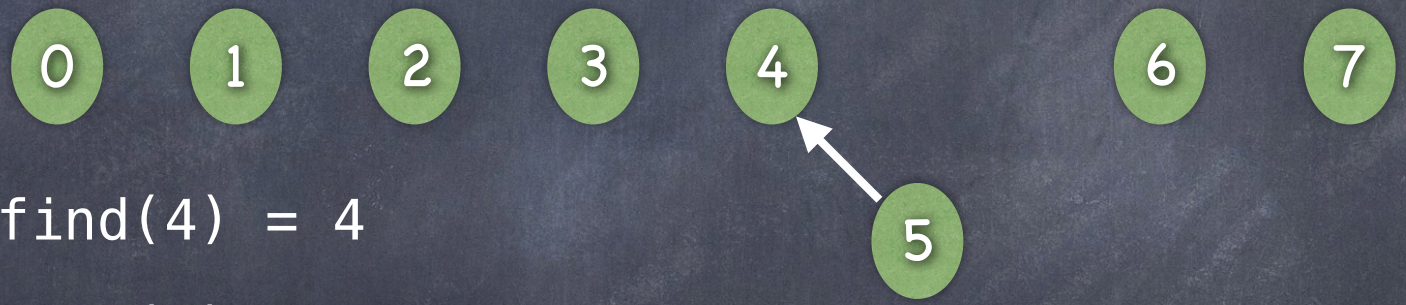
- Union(i, j): crea una nueva clase de eq. con todos los elementos de las clases i y j
- Find(i) devuelve el representante de la clase i



Disjoint sets



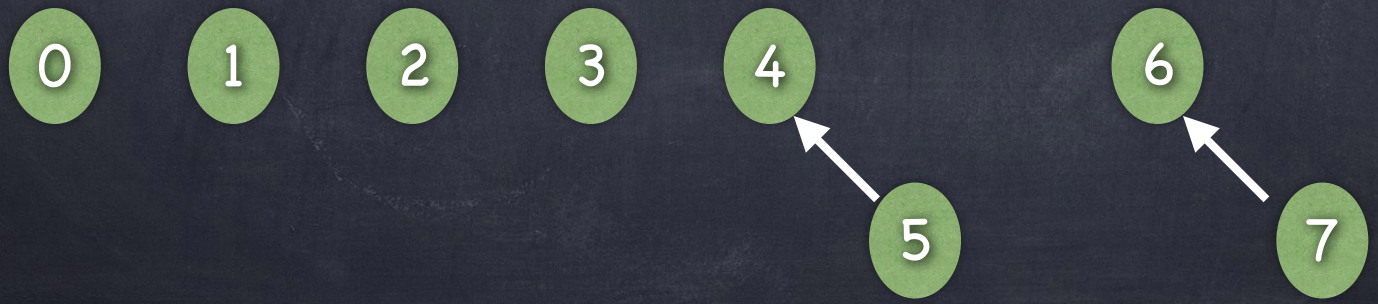
union(4,5)



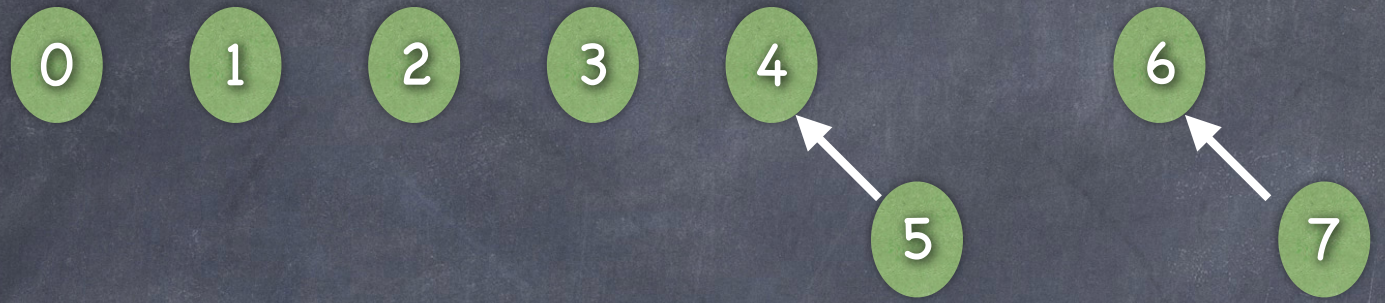
find(4) = 4

find(5) = 4

union(6,7)

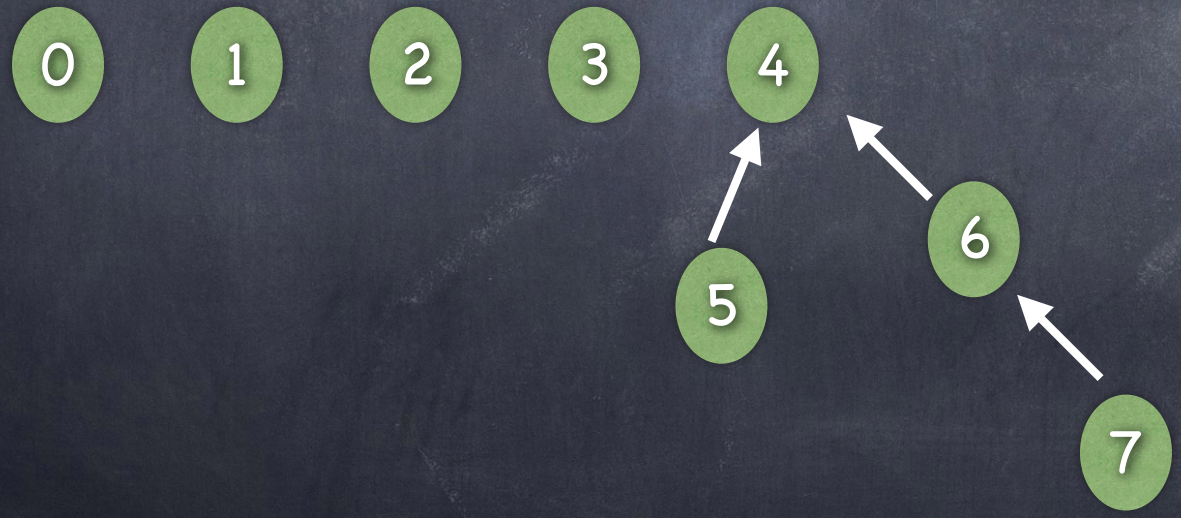


Disjoint sets




```
find(6) = 6
find(7) = 6
```

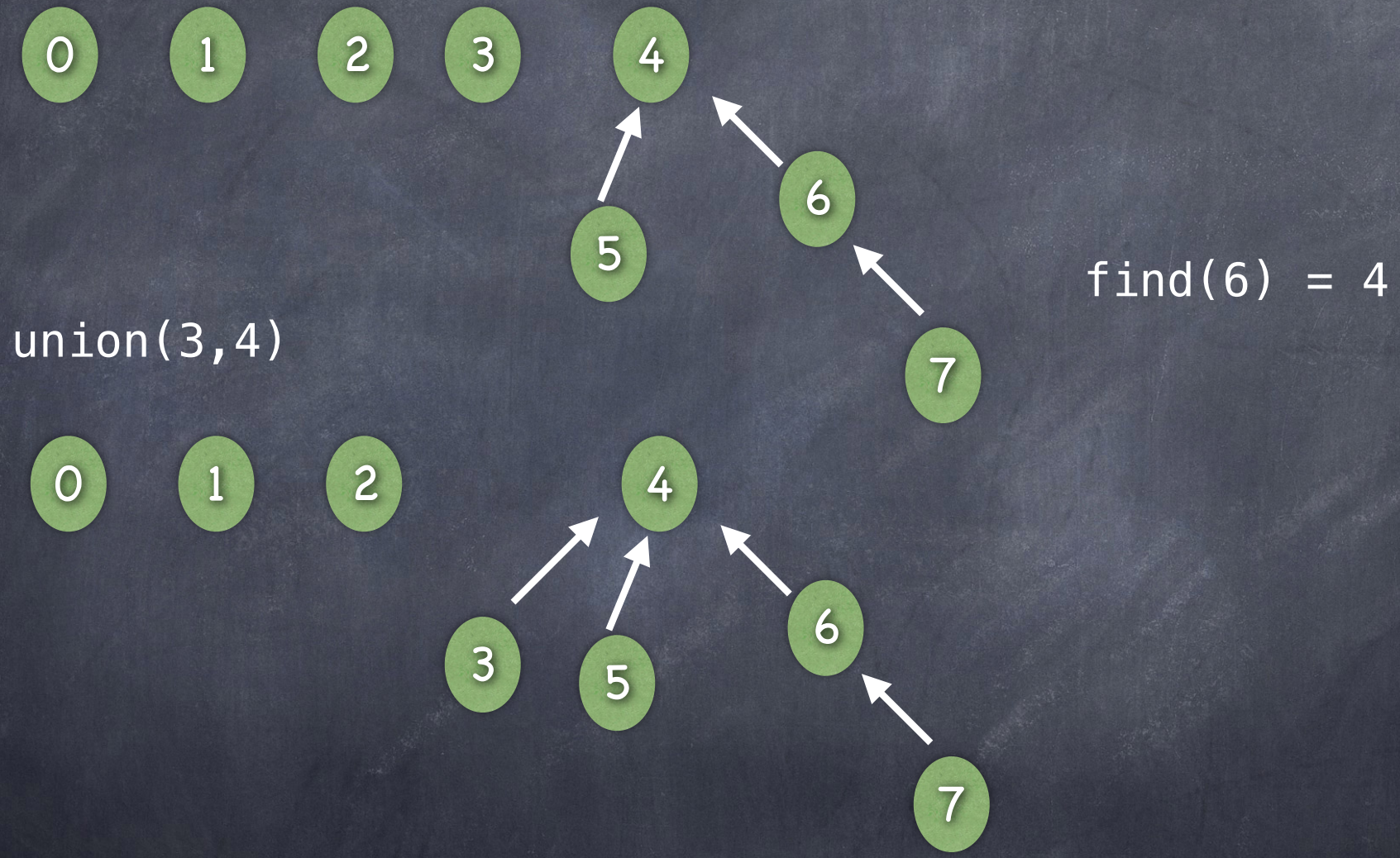
```
union(6,4)
```




```
find(6) = 4
```

find y union $\in O(logV)$

Disjoint sets



find y union $\in O(logV)$

Disjoint set

Estructura en memoria: Son árboles invertidos (los hijos apuntan al padre) implementados de mediante un **arreglo de enteros**

No son árboles binarios ni ordenados: Son árboles **N-arios** (un padre puede tener infinitos hijos apuntándole en simultáneo) y no tienen orden de menor/mayor.

Disjoint set

Por qué las operaciones $\in O(log\ V)$

Union: se cuelga el árbol de menor altura bajo la raíz del de mayor altura. Se garantiza matemáticamente que la altura máxima del bosque sea $log\ V$.
Por eso, cualquier búsqueda (find) hacia arriba para buscar al representante toma tiempo logarítmico.

Ejemplo: Árbol Abarcador de Costo Mínimo (cont.)

```
MST-KRUSKAL(G)
  A = ∅

  for each vertex v ∈ G.V
     MAKE-SET(v)
  sort the edges of G.E into nondecreasing order by weight

  for each edge (u, v) ∈ G.E, taken in nondecreasing order by weight
     if FIND-SET(u) ≠ FIND-SET(v)
        A = A ∪ {(u, v)}
        UNION(u, v)

  return A
```

O(V)

O(E log E)

Se ejecuta E veces

O(log V)

O(log V)

Ejemplo: Árbol Abarcador de Costo Mínimo (cont.)

$$T(V,E) = V + E\,.\,log\ E + E\,.\,log V$$

el número de aristas E está acotado por $V^2 (E \leq V^2)$

$$\log E \leq \log(V^2) \implies \log E \leq 2 \log V$$

$$T(V,E) \leq V + E\,.\,log\ V + E\,.\,log V$$

$$T(V,E) \in O(E\,.\,log V)$$

Algoritmo de Dijkstra

El algoritmo de Dijkstra computa caminos de costo mínimo desde un nodo origen al resto de los nodos.

Dijkstra: caminos de costo mínimo desde un origen

Entrada: un grafo con pesos no negativos y un vértice origen v.

Salida: para cada vértice x del grafo:

`dist[x]`: el costo del camino mínimo de v a x.

`prev[x]`: el vértice que viene justo antes de x en ese camino.

Algoritmo de Dijkstra

¿Qué calcula el algoritmo?

`dist[x]`: la distancia mínima conocida de v a x. Arranca en ∞, salvo `dist[v] = 0`.

`prev[x]` (**previo**): el predecesor de x en el camino mínimo. Arranca en `null`.

`dist` nos dice **cuánto** cuesta llegar.

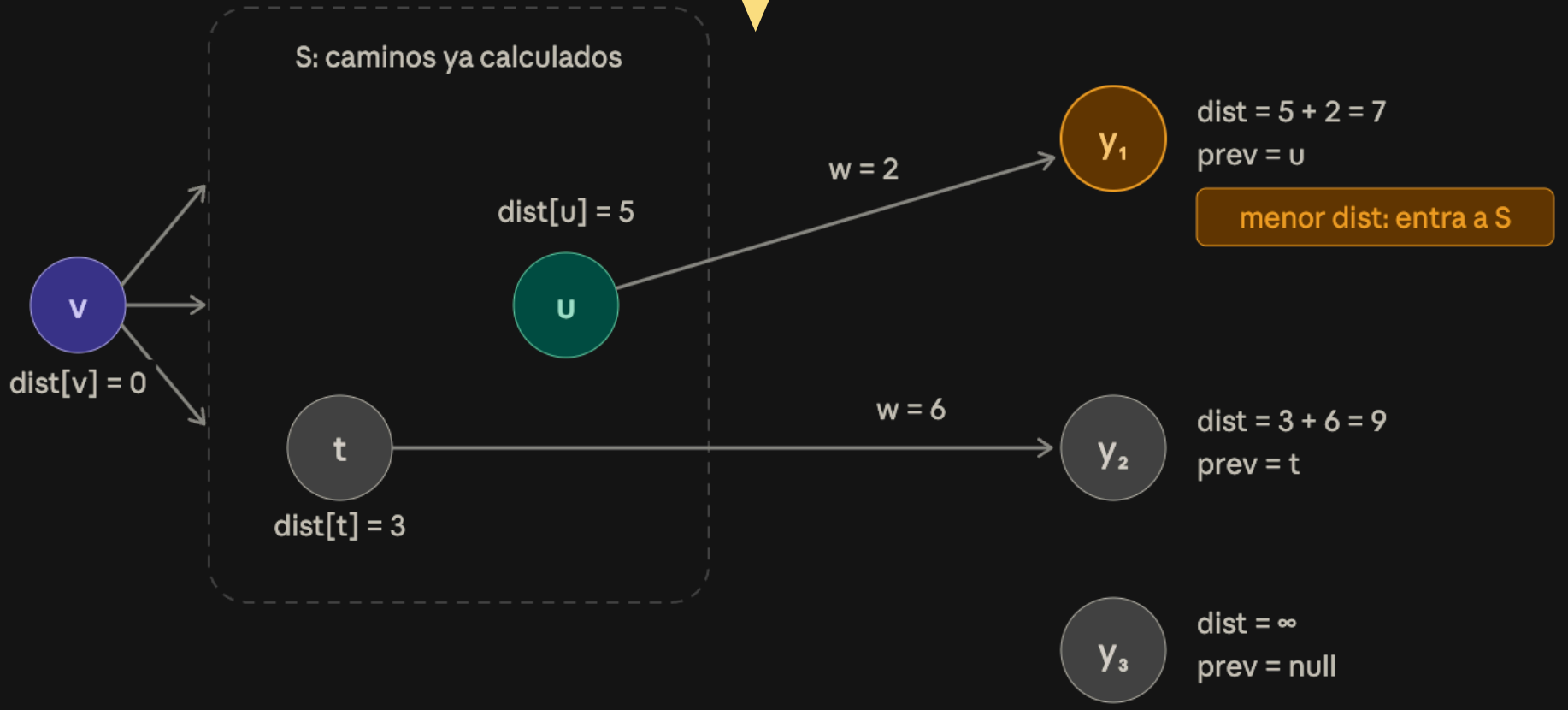
`prev` nos dice **por dónde** ir.

Reconstruir el camino a x: se sigue **prev** hacia atrás hasta llegar a *v*:
x → prev[x] → prev[prev[x]] → ... → v

Condición: los pesos de las aristas deben ser ≥ 0.

Algoritmo de … a idea

El próximo en entrar a S es el nodo más cercano a v **usando un camino que pasa solo por nodos de S**



1) Relajar: se actualizan dist y prev de los vecinos

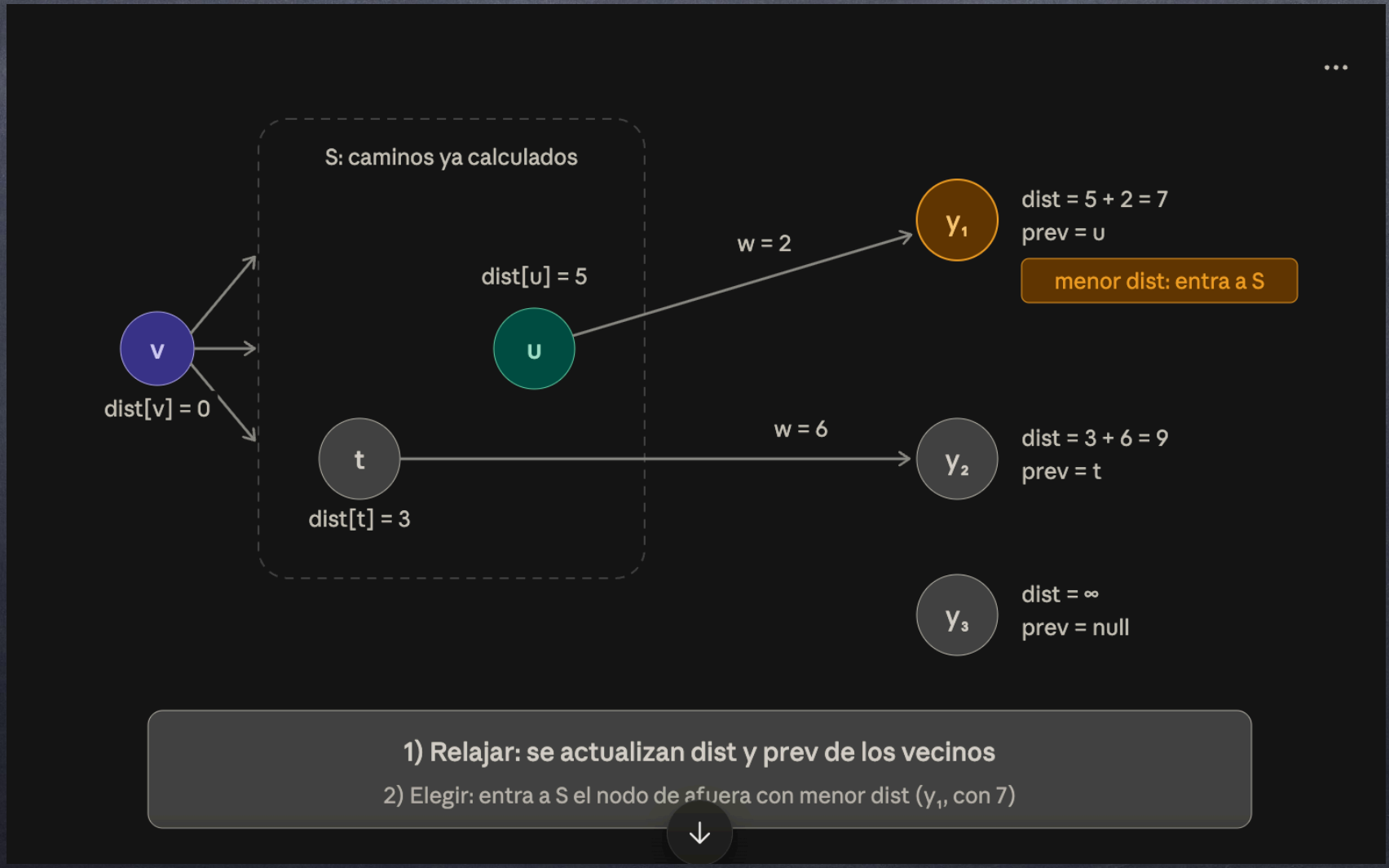
2) Elegir: entra a S el nodo de afuera con menor dist (y_1, con 7)

↓

Algoritmo de ea

calcula el camino de cos mínimo desde un origen a todos los demás vértices.

En cada paso se agrega el vértice de menor `dist` que todavía no está en S (**estrategia greedy**)



Algoritmo

```
public void dijkstra(Graph g, Vertex v, Vertex[] prev, int[] dist){
    for all x in g.vertices() do{
    // para todos los vertices del grafo
        prev[x] = null; // el previo es null
        dist[x] = inf; // la dist. mas corta el inf.
    }
    dist[v] = 0; // la dist. al origen es 0
    s = new Set(); // s conj. vacio
    // una cola de prioridad con los vertices,
    // donde la clave de cada u es dist[u]
    q = new PriorityQueue(g.vertices());
    ...

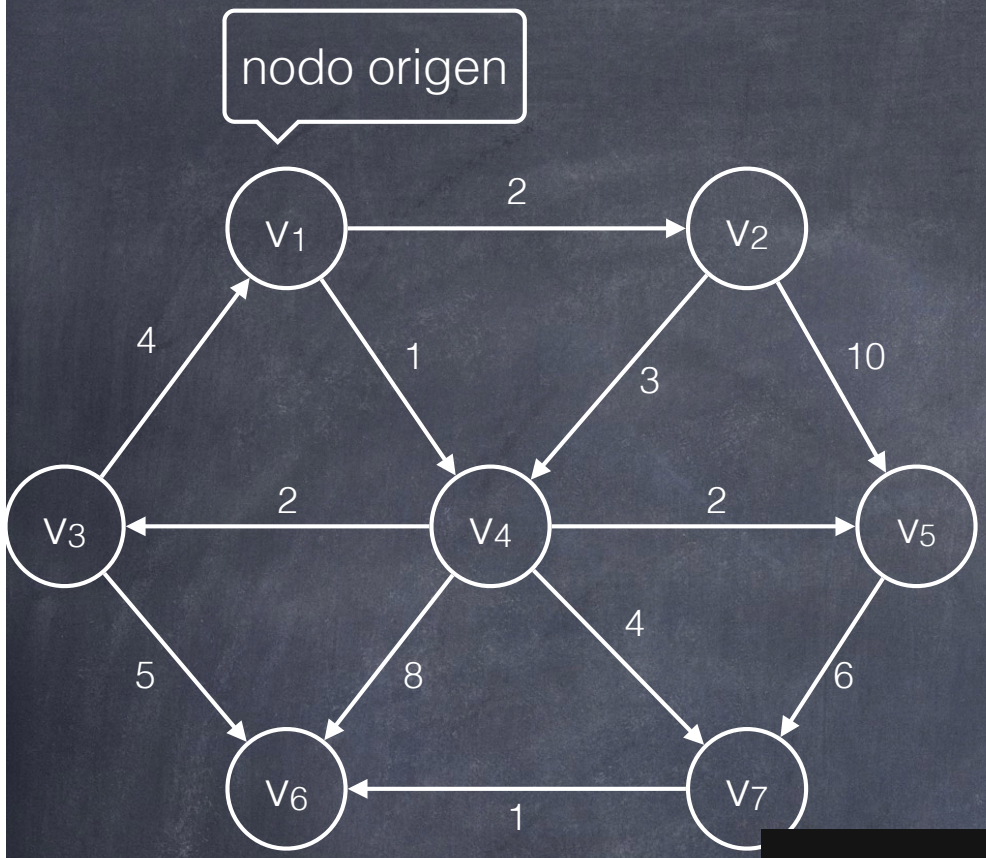

}
```

Algoritmo




```
while (!q.isEmpty()){
    u = q.extractMin(); // se saca el mas cercano
    s.insert(u); // se lo pone en S
    for each edge (u,y) where y is not in s do{
    // se recalculan la distancias de los vecinos
        if (dist[u] + w(u,y) < dist[y]){
            dist[y] = dist[u] + w(u,y);
            prev[y] = u;
            // se actualiza la cola de prioridad
            q.decreaseKey(y,dist[y]);
        }
    }
}
```

Ejemplo



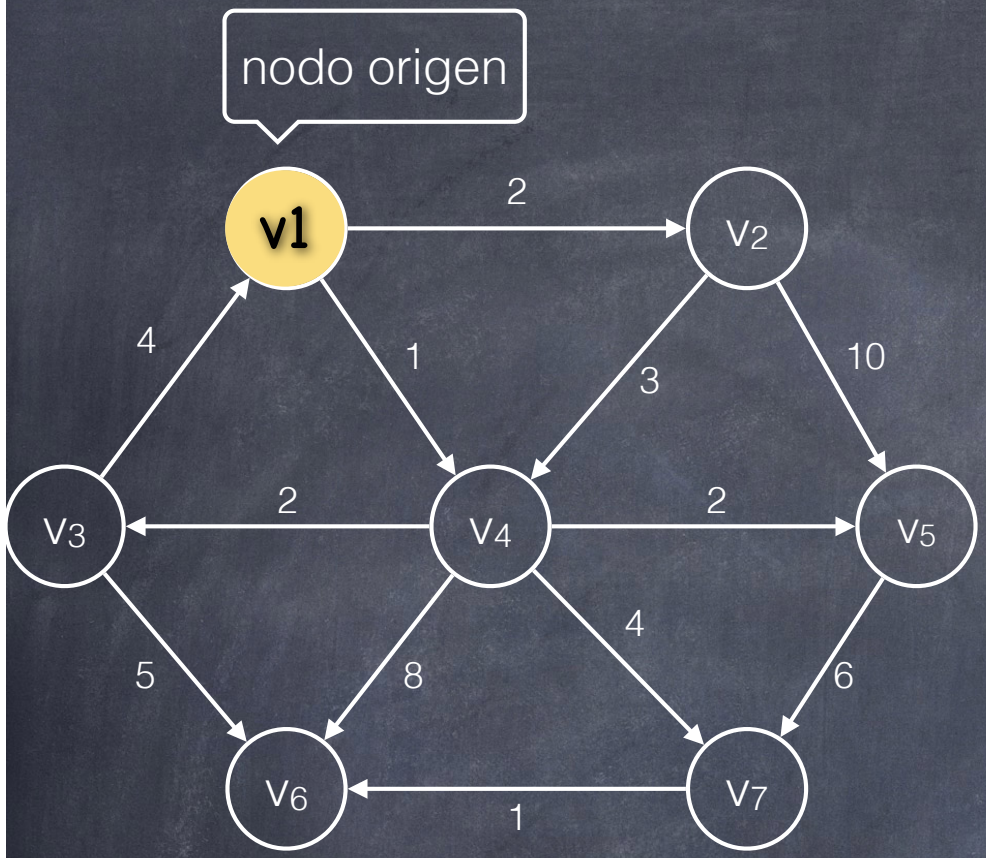
S = { }

vértice	v1	v2	v3	v4	v5	v6	v7
dist	0	∞	∞	∞	∞	∞	∞
prev	null	null	null	null	null	null	null

q clave: dist	v1: 0	v2: ∞	v3: ∞	v4: ∞	v5: ∞	v6: ∞	v7: ∞

frente de la cola: el de menor dist (v1)

Ejemplo

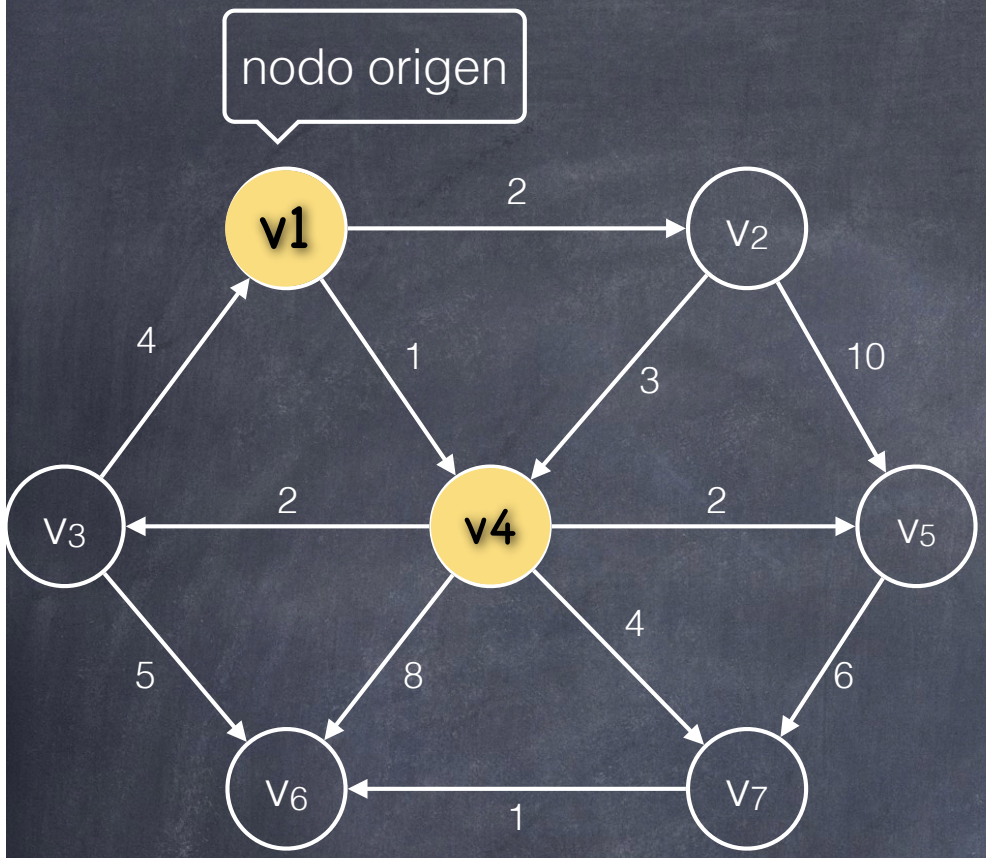


S= {1}

vértices	v1	v2	v3	v4	v5	v6	v7
dist	0	2	inf	1	inf	inf	inf
prev	null	1	null	1	null	null	null

q: v4= 1 | v2= 2 | v3= inf | v5= inf | v6= inf | v7= inf

Ejemplo



S= {1,4}

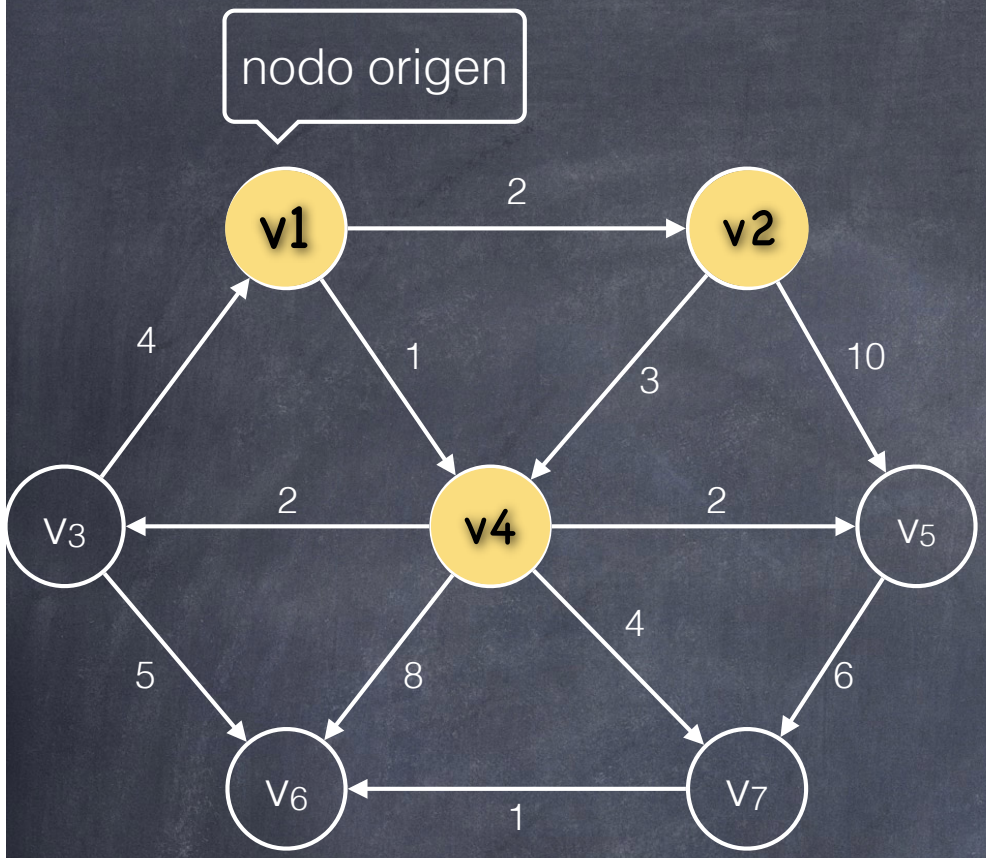
vértices	v1	v2	v3	v4	v5	v6	v7
dist	0	2	3	1	3	9	5
prev	null	1	4	1	4	4	4

q: v2= 2 | v3= 3 | v5= 3 | v7= 5 | v6= 9

extractMin: sale v4 (dist 1) y entra a S.
Relajar vecinos de v4 (v3, v5, v6, v7; v1 ya está en S):

- v3: 1+2 = 3 < inf → `dist=3, prev=4`
- v5: 1+2 = 3 < inf → `dist=3, prev=4`
- v6: 1+8 = 9 < inf → `dist=9, prev=4`
- v7: 1+4 = 5 < inf → `dist=5, prev=4`

Ejemplo



S= {1, 4, 2}

vértices	v1	v2	v3	v4	v5	v6	v7
dist	0	2	3	1	3	9	5
prev	null	1	4	1	4	4	4

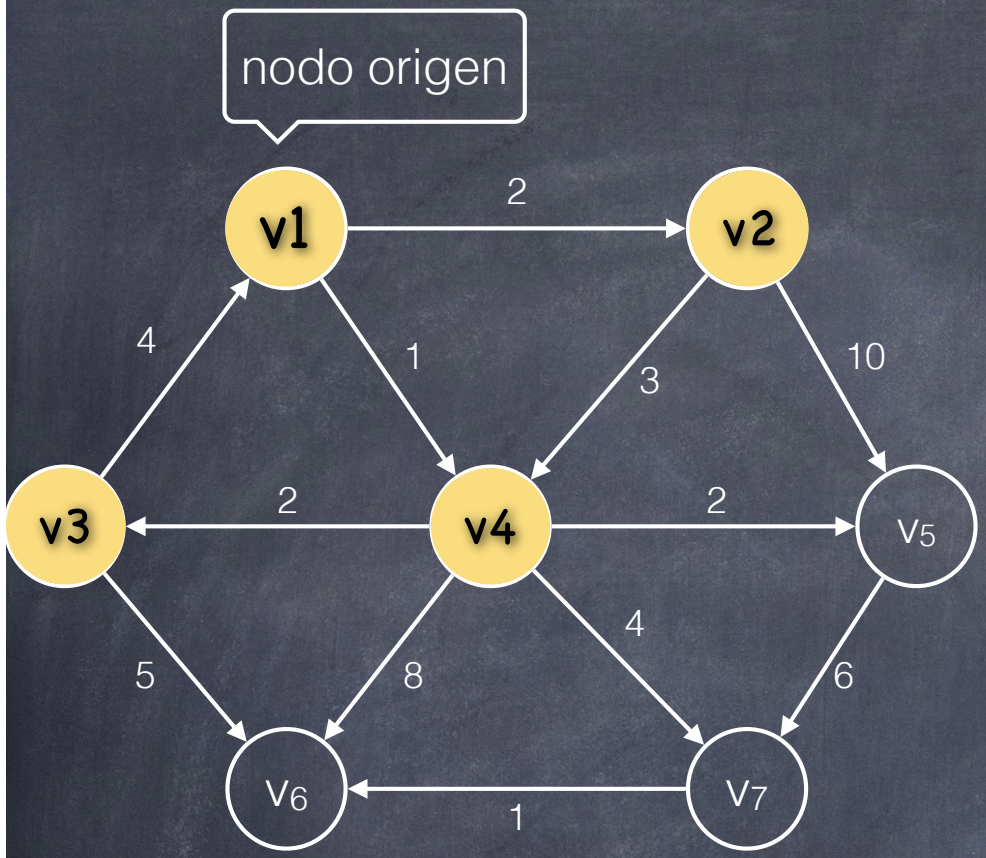
q

v3= 3 | v5= 3 | v7= 5 | v6= 9

extractMin: sale v2 (dist 2) y entra a S.
Relajar vecinos de v2:

- v4: ya está en S, se ignora.
- v5: 2+10 = 12, no mejora el 3 actual → **sin cambios**

Ejemplo



S= {1, 4, 2, 3}

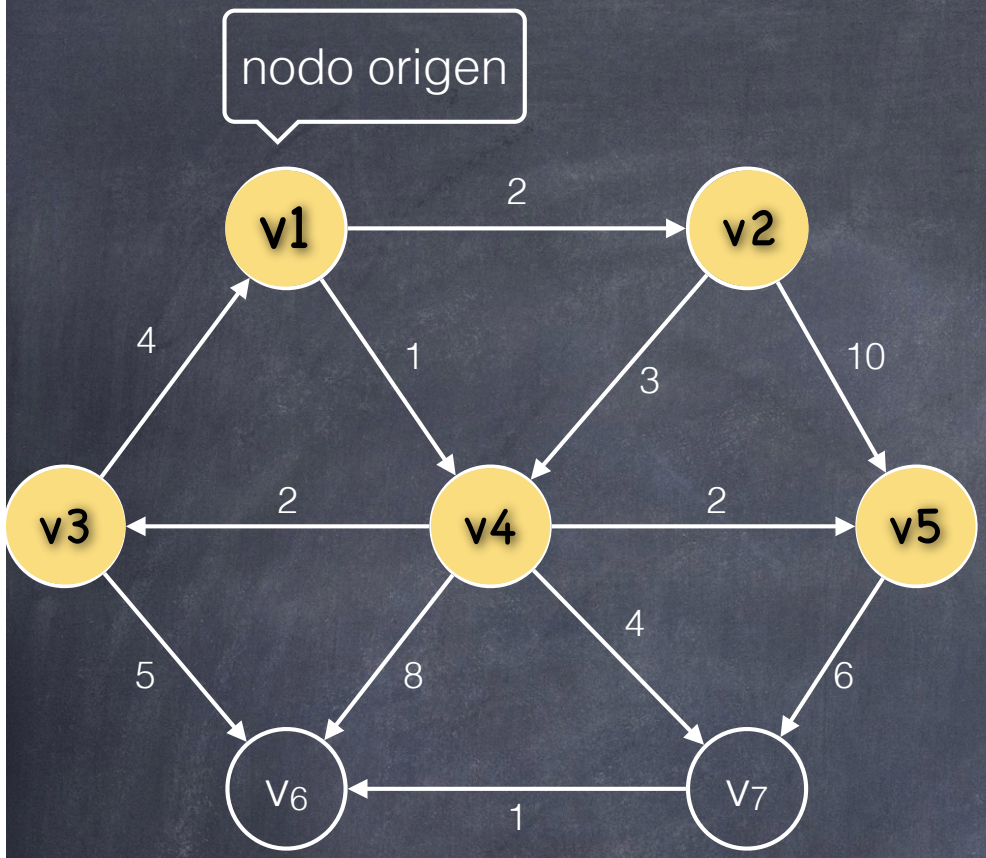
vértices	v1	v2	v3	v4	v5	v6	v7
dist	0	2	3	1	3	8	5
prev	null	1	4	1	4	3	4

q: v5= 3 | v7= 5 | v6= 8

extractMin: sale v3 (dist 3) y entra a S.
Relajar vecinos de v3:

- v1: ya está en S, se ignora.
- v6: 3+5 = 8 < 9 → `dist=8`, `prev=3`

Ejemplo



S= {1, 4, 2, 3, 5}

vértices	v1	v2	v3	v4	v5	v6	v7
dist	0	2	3	1	3	8	5
prev	null	1	4	1	4	3	4

q

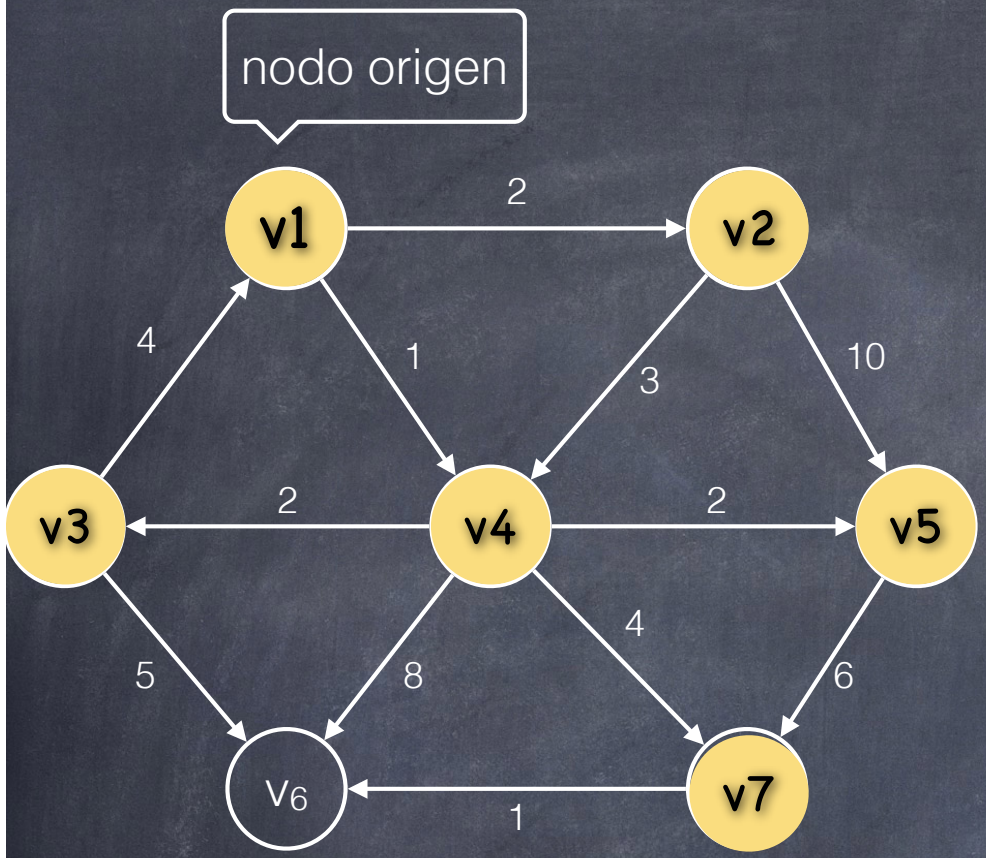
v7= 5

v6= 8

extractMin: sale v5 (dist 3) y entra a S.
Relajar vecinos de v5:

- v7: 3+6 = 9, no mejora el 5 actual → **sin cambios**

Ejemplo



S= {1, 4, 2, 3, 5, 7}

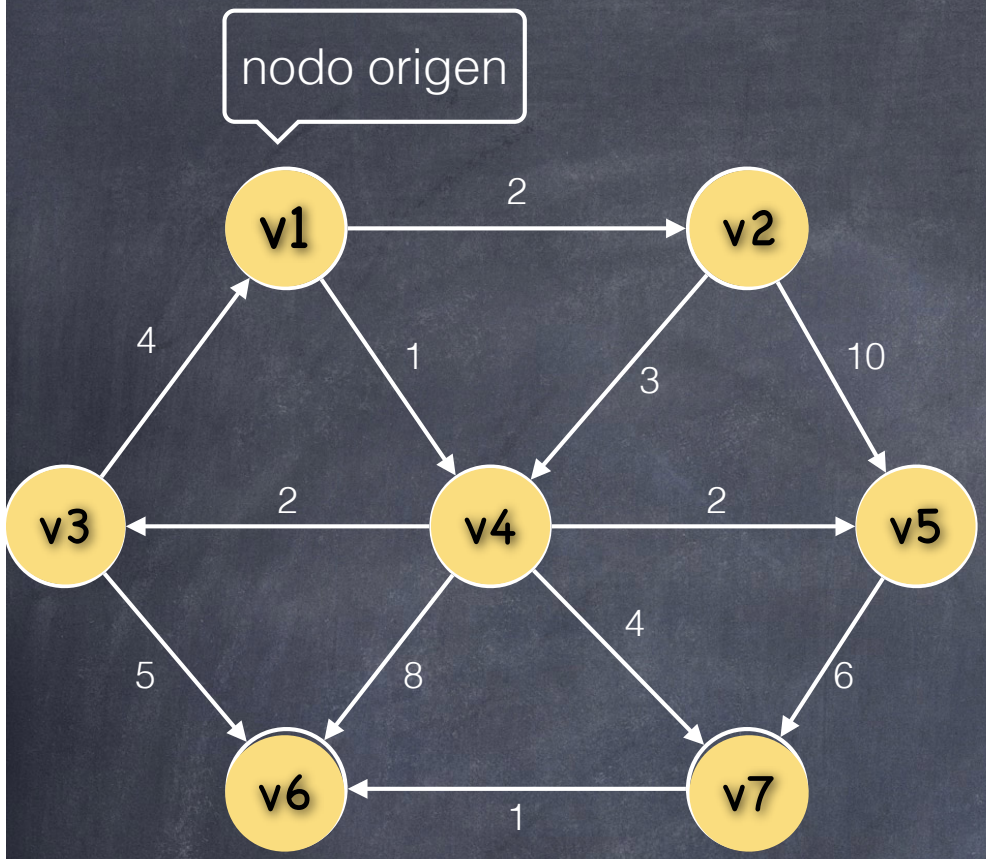
vértices	v1	v2	v3	v4	v5	v6	v7
dist	0	2	3	1	3	6	5
prev	null	1	4	1	4	7	4

q v6= 6

extractMin: sale v7 (dist 5) y entra a S.
Relajar vecinos de v7:

- v6: 5+1 = 6 < 8 → `dist=6, prev=7`

Ejemplo



S= {1, 4, 2, 3, 5, 7, 6}

vértices	v1	v2	v3	v4	v5	v6	v7
dist	0	2	3	1	3	6	5
prev	null	1	4	1	4	7	4

q {}

extractMin: sale v6 (dist 6) y entra a S.
Relajar vecinos de v6: no tiene aristas salientes, no hay nada que hacer.
q queda vacía y el algoritmo termina.

Algoritmo

Veamos el pseudocódigo:

```
public void dijkstra(Graph g, Vertex v, Vertex[] prev, int[] dist){
    for all x in g.vertices() do{
    // para todos los vertices del grafo
        prev[x] = null; // el previo es null
        dist[x] = inf; // la dist. mas corta el inf.
    }
    dist[v] = 0; // la dist. al origen es 0
    s = new Set(); // s conj. vacio
    // una cola de prioridad con los vertices, donde la clave de cada u es dist[u]
    q = new PriorityQueue(g.vertices());
    while (!q.isEmpty()){
        u = q.extractMin(); // se saca el mas cercano
        s.insert(u); // se lo pone en S
        for each edge (u,y) where y is not in S {
        // se recalculan la distancias de los vecinos
            if (dist[u] + w(u,y) < dist[y]){
                dist[y] = dist[u] + w(u,y);
                prev[y] = u;
                q.decreaseKey(y,dist[y]); // se actualiza la cola de prioridad
            }
        }
```

O(V)

Se ejecuta V veces

O(log V) con heaps

Se ejecuta E veces al total

O(log V) con heaps

Ejemplo: Árbol Abarcador de Costo Mínimo (cont.)

$$T(V,E) = V + V\ log\ V + E\ .\ logV =$$

$$V + (V + E)\ log\ V$$

Asumiendo $(E \geq V - 1)$

$$T(V,E) \in O(E\ .\ logV)$$

Corrección

No llegamos hasta acá: próxima clase

El invariante del algoritmo es:

Distancia más corta de v a s

$$\langle \forall s \in S :: dist[s] = \delta(v, s) \rangle$$

Para todos los nodos en S la distancia calculada en d[s] es la del camino más corto de v a s

Idea de la prueba: Suponer que el invariante no se cumple en algún paso, es decir, agregamos un nodo w a S para el cual no tenemos en d[w] la distancia más corta, luego por propiedades de los caminos más corto y la forma de actuar del algoritmo nos dan una contradicción.

Corrección

Al inicio del Algoritmo:

Distancia más corta de v a s

$$\langle \forall s \in S :: dist[s] = \delta(v, s) \rangle$$

$$S = \emptyset, \quad dist[v] = 0, \quad dist[w] = \infty \ \forall w \neq v$$

Como S está vacío, el invariante:

$$\forall s \in S :: dist[s] = \delta(v, s)$$

El invariante es verdadero antes del primer ciclo.

Corrección

Supongamos que al inicio de una iteración cualquiera del bucle `while`, el invariante se cumple:

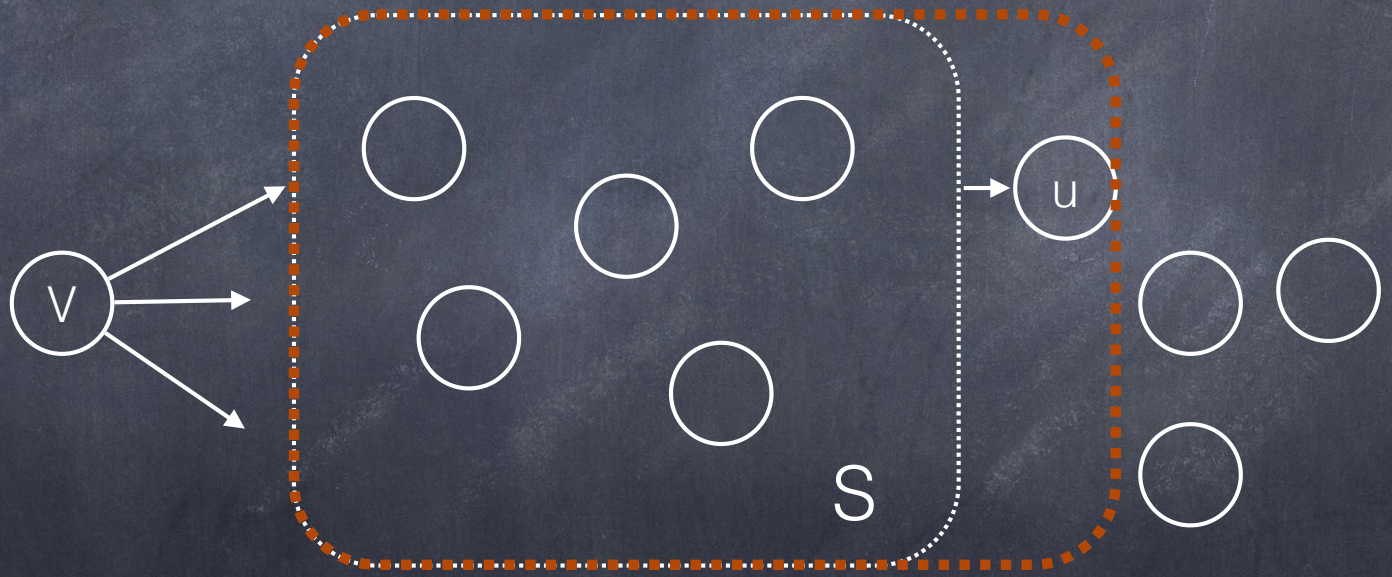
$$\forall s \in S :: dist[s] = \delta(v, s)$$

Queremos demostrar que después de agregar un nuevo vértice u a S, el invariante sigue siendo cierto.

Corrección

Supongamos que el algoritmo **falla** en algún punto. Es decir, existe un **primer vértice u** que se añade a S y para el cual:

$$dist[u] \neq \delta(v, u)$$



Como sabemos que
$dist[u] \geq \delta(v, u)$, necesariamente:

$$dist[u] > \delta(v, u)$$

Corrección

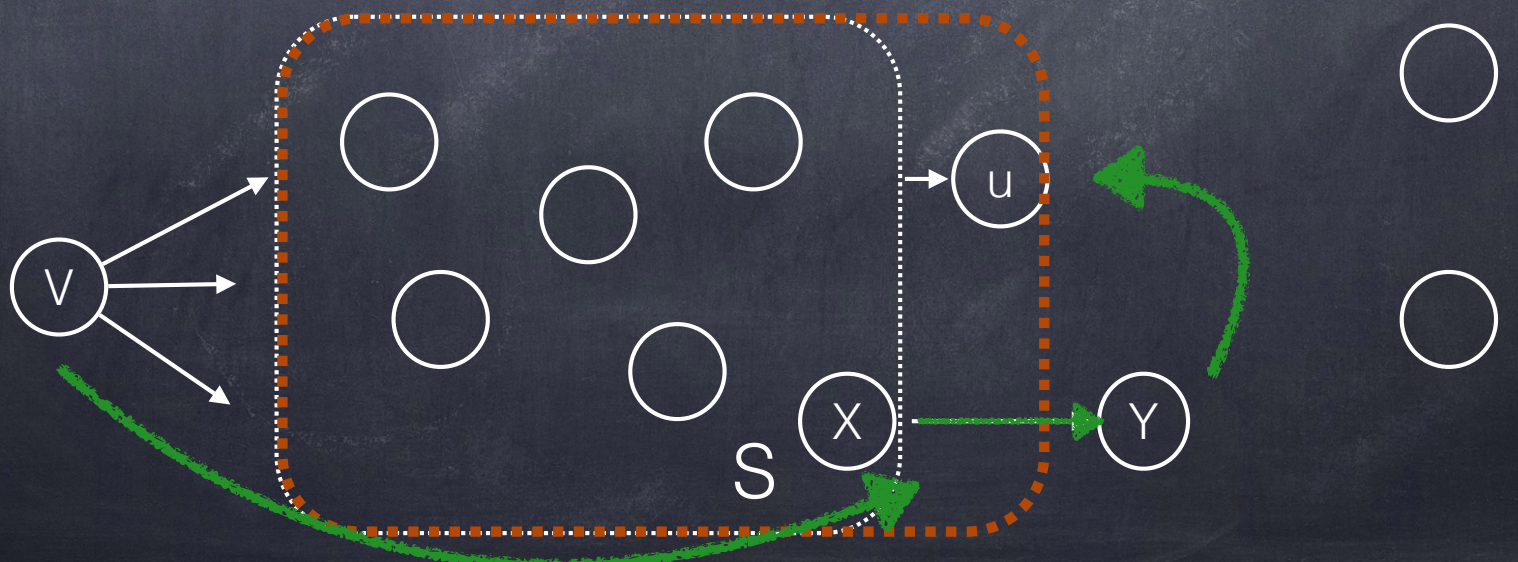
Sea $P = v \rightsquigarrow u$

un camino más corto desde v hasta u, de costo $|P| = \delta(v, u)$

En este camino, sea y el primer vértice de P que no pertenece a S, y sea x su predecesor inmediato en P

$$x \in S, \quad y \notin S, \quad (x, y) \in E$$

Entonces:



Corrección

Por la definición del camino más corto:

$$\delta(v, y) = \delta(v, x) + w(x, y)$$

Como $x \in S$ y el invariante se cumple para todos los vértices en S:

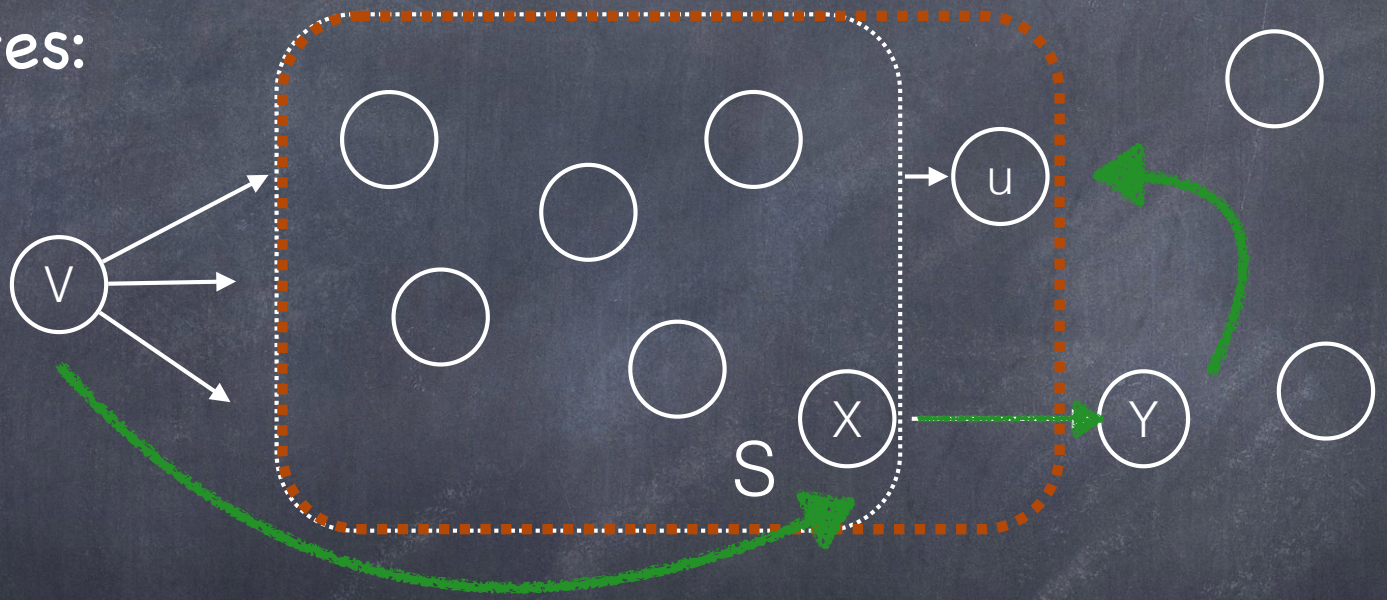
$$dist[x] = \delta(v, x)$$

Corrección

Si $P = v \rightsquigarrow u$

El camino más corto desde v hasta u, de costo $|P| = \delta(v, u)$

Entonces:



$v \rightsquigarrow x \rightarrow y$ es el camino más corto de v a y

$$dist[y] = \delta(v, y)$$

Corrección

Además, el algoritmo selecciona siempre el vértice u con la **distancia mínima** entre todos los vértices que no están en S:

$$dist[u] \leq dist[z], \quad \forall z \notin S$$

En particular, como $y \notin S$, $dist[u] \leq dist[y]$

$$dist[y] = \delta(v, y) \leq \delta(v, u) < dist[u]$$

$$\Longrightarrow dist[y] < dist[u]$$

Contradicción!